


**TRABAJO INTEGRADOR FINAL**

Análisis de Sistemas

**INTEGRANTES DEL EQUIPO**

Carballo Tobías

Dacuy Santino

Pereyra Lucrecia

**CÁTEDRA**

Taller de Integración

**EQUIPO DOCENTE**

Esp. Lic. Escalante Julián

Lic.  Figueroa Daniel

**FECHA**

24/04/2026

## PROBLEMA Y ALCANCE.

**Problema:**

La empresa “Mate Único”, fundada en el año 2022, se dedica exclusivamente a la venta minorista de mates y productos relacionados, tales como bombillas y bolsos materos. Actualmente enfrenta la necesidad de expandirse en el mercado. Al contar con una oferta que incluye mates de diversos materiales y combos predefinidos, la empresa requiere una solución tecnológica que le permita digitalizar su catálogo, gestionar su inventario y automatizar sus procesos de ventas para poder alcanzar a un público más amplio mediante una plataforma propia.

**Alcance:**

El proyecto comprende el diseño y desarrollo de un e-commerce orientado exclusivamente a la venta minorista. El sistema administrará un catálogo digital donde los mates individuales y los combos (compuestos por mate + bombilla de alpaca, o mate + bombilla de alpaca + bolso matero) serán gestionados como productos independientes, aplicando un estricto control de inventario en tiempo real que bloqueara las transacciones y mostrará avisos en caso de que el stock sea insuficiente.

En cuanto a los usuarios, la navegación por el catálogo será pública, pero el acceso a las funcionalidades de compra y valoraciones de productos requerirá la autenticación obligatoria del cliente mediante OAuth de Google. Por otro lado, se desarrollará un panel de control exclusivo para el administrador, el cual centralizará la gestión de altas, bajas y modificaciones del catálogo, la administración de promociones o cupones de descuento, y la visualización de métricas y estadísticas clave del negocio, tales como facturación y productos más vendidos.

El procesamiento de las transacciones monetarias se realizará integrando de manera exclusiva la API de Mercado Pago. Finalmente, la logística de envíos a

domicilio será gestionada por proveedores externos (Andreani, OCA o Correo Electrónico), calculando los costos de forma automática basándose en el código postal y la provincia del comprador.

## REQUERIMIENTOS FUNCIONALES Y NO FUNCIONALES.

**Requerimientos funcionales:**

Para la definición de las funcionalidades del sistema se ha utilizado la técnica de priorización MoSCoW, determinando el nivel de importancia de cada requerimiento:

Must Have (Esenciales):

- El sistema debe mostrar un catálogo que incluya el nombre, la descripción, fotos, material y capacidad como atributos mínimos de cada producto.
- El sistema debe tener un mate como producto único.
- El sistema debe gestionar combos como productos independientes:
    - mate + bombilla,
    - mate + bombilla + bolso.
- El sistema debe aplicar las siguientes restricciones de negocio:
    - Bombillas y bolsos no se venden por separado.
    - Único material de bolso disponible.
    - Modelo único de bombilla de alpaca.
    - Varios tipos de mates en cuatro materiales distintos: calabaza, madera, vidrio y metal.
- El sistema debe gestionar el stock en tiempo real.
- El sistema debe mostrar mensajes de error si el stock no es suficiente, por ejemplo, en el caso de que existan 5 productos y se soliciten 6.
- El sistema debe aceptar solo venta minorista.
- El usuario debe estar logueado obligatoriamente para poder comprar.
- El sistema debe autenticar al usuario mediante Google (OAuth).

- La cuenta del usuario debe incluir: nombre, apellido, dirección, código postal, teléfono.
- El usuario debe poder editar solo la dirección en el perfil del cliente.
- El sistema debe aceptar solo Mercado Pago como medio de pago.
- El sistema debe integrarse con la API de Mercado Pago + webhooks para actualización de estados.
- El sistema debe ofrecer sólo envío a domicilio mediante gestión externa con las empresas Andreani, OCA o Correo Argentino.
- El sistema debe calcular el costo de envío según una tabla fija por código postal y provincia.
- El sistema debe enviar al usuario un comprobante mediante email con los datos esenciales de la compra.
- El sistema debe permitir tracking de envío a través de un servicio externo enviado manualmente al cliente.
- El administrador debe contar con un panel para gestionar productos, stock, pedidos y cupones.
- El sistema debe registrar eventos importantes mediante un sistema de logging.
- El usuario debe poder agregar productos al carrito.
- El usuario debe poder aumentar o disminuir cantidades de los productos en el carrito.
- El usuario debe poder eliminar productos del carrito.
- El sistema debe dar aviso de bajo stock.
- El sistema debe impedir que se agregue una cantidad mayor a la disponible.

Should Have (Importantes pero no críticos):

- El sistema debería limitar la cantidad y calidad de fotos por producto.

- El usuario debería poder aplicar filtros al catálogo según el material, el color y el precio.
- El usuario debería poder dejar reseñas de productos, permitiendo esta acción únicamente a compradores verificados.
- El sistema debería mostrar productos sugeridos en cada página de producto.
- El usuario debería tener un carrito persistente entre sesiones.
- El usuario debería ver el carrito siempre visible en la UI.
- El usuario debería tener acceso a una barra de búsqueda global de productos.
- El sistema deberia mostrar algún banner en la home para usuarios inactivos.
- El usuario debería poder aplicar cupones de descuento basados en un porcentaje.
- El sistema debería validar reglas y restricciones de los cupones, tales como su vigencia, cantidad de usos y exclusividad.
- El administrador debería poder consultar reportes básicos sobre ventas, clientes y los productos más vendidos.
- El administrador debería poder realizar la carga manual de tracking de envíos.
- El sistema debería enviar notificaciones por email tras la compra.
- El sistema debería enviar promociones por email a usuarios registrados.

Could Have (Deseables pero prescindibles):

- El usuario podría tener una vista previa de grabado personalizado en mates.
- El sistema podría generar etiquetas automáticas en catálogo, tales como “Nuevo” o “Más vendido”.

- El sistema podría enviar emails de marketing para aquellos usuarios que hayan aceptado el consentimiento previo u opt-in.
- El usuario podría contar con una búsqueda tolerante a errores que incluya autocorrección y sugerencias.
- El administrador podría contar con un dashboard avanzado que visualice el top de clientes, el top de productos y diversas métricas gráficas.
- El sistema podría mostrar un footer con disponibilidad de redes sociales.
- El usuario podría acceder a soporte al cliente por teléfono.

Won't Have (Excluidos por ahora):

- El usuario no podrá realizar compras como invitado, por lo que será obligatorio realizar el proceso de registro previo.
- El sistema no ofrecerá retiro en sucursal o tienda física.
- El sistema no integrará gestión de redes sociales en la plataforma.
- El sistema no ofrecerá medios de pago alternativos a Mercado Pago.
- El sistema no ofrecerá variantes de bombillas individuales.
- El sistema no ofrecerá bolsos de distintos materiales.

**Requerimientos no funcionales:**

De acuerdo a las restricciones técnicas y arquitectónicas exigidas para el proyecto, el sistema deberá cumplir con lo siguiente:

- Arquitectura y diseño web: La solución debe desarrollarse como una aplicación web particionada en capas, contando con una interfaz (front-end) obligatoriamente responsive. Para su construcción visual se empleará un toolkit de interfaz y un framework de desarrollo del lado del cliente.
- Back-End: El servidor de la aplicación debe implementar una API que utilice un ORM (Mapeo Objeto-Relacional) o la tecnología GraphQL para gestionar la interacción con la base de datos.

- Base de datos: Los datos del sistema deben persistir en un sistema gestor de bases de datos relacionales SQL, como PostgreSQL, MySQL o SQLite.
- Control de versiones y seguridad: Todo el código fuente deberá gestionarse mediante un sistema de control de versiones utilizando Git. Asimismo, la autenticación deberá delegarse a redes sociales, y se requerirá la implementación de un sistema de logging adecuado.

## WIREFRAMES.

El diseño de las interfaces del e-commerce “Mate Único” ha sido proyectado para ofrecer una experiencia de usuario fluida, intuitiva y visualmente atractiva, estructurada en dos perfiles de acceso: cliente y administrador. Para garantizar la consistencia visual, todas las pantallas, a excepción del login, mantienen un header y un footer constantes que facilitan la navegabilidad y el acceso a la información de contacto.

**Vistas del lado del cliente.**

- Inicio de sesión (Login): En esta pantalla, los usuarios pueden iniciar sesión únicamente mediante su cuenta de Google. La interfaz cuenta con un espacio a la izquierda para una imagen representativa y en la parte superior derecha otra imagen decorativa. Debajo se encuentran los iconos de nuestras redes sociales para facilitar conexión con la marca. Este diseño busca una experiencia simple y directa, asegurando que el acceso al sistema sea rápido y seguro. *Enlace a la vista: Login.*
- Pantalla principal (Home): La pantalla da la bienvenida al usuario con un mensaje de saludo y una imagen destacada del producto. Se muestran tres productos destacados que, al presionar “Ver más”, llevan a la vista completa del catálogo. A la izquierda permite filtrar por categorías y visualizar los mates más vendidos. Además cuenta con un carrusel

horizontal de imágenes promocionales y un bloque de texto que explica cómo ayudamos a encontrar el mate ideal. *Enlace a la vista: Home.*

- Filtro por producto: En esta vista el usuario puede aplicar filtros según características específicas como material, color y tipo de producto. Se muestran únicamente los productos que cumplen con los criterios seleccionados, permitiendo una búsqueda rápida y personalizada. *Enlace a la vista: Filtro.*
- Detalle de producto: Página que muestra los detalles de un producto específico, incluyendo cuatro imágenes en formato cuadrado. Habilita la opción de añadir a favoritos, agregar un grabado, seleccionar cantidad y color, y añadir al carrito. En la parte inferior, se muestran las calificaciones de otros compradores y productos similares. *Enlace a la vista: Detalle producto.*
- Perfil de usuario: El perfil del cliente está dividido en dos secciones. A la izquierda la foto de perfil, una descripción pequeña, el historial de compras y una lista de favoritos. A la derecha, un formulario para modificar la información personal, un botón para guardar los cambios y la opción de actualizar la imagen de perfil. *Enlace a la vista: Perfil usuario.*
- Carrito: Accesible desde el encabezado. Aquí se muestran todos los productos seleccionados, brindando la posibilidad de eliminarlos. Al presionar “Confirmar”, se procede al paso de dirección y pago. *Enlace a la vista: Carrito.*
- Pago - Dirección: El usuario completa su información de contacto )nombre, apellido, dirección, piso, código postal, provincia y ciudad). A la derecha se muestra un resumen del carrito con opciones para agregar cupones, y el cálculo detallado del subtotal, costo del grabado, envío, descuento y total. *Enlace a la vista: Pago - Dirección.*

- Envío: El usuario selecciona la modalidad de entrega a domicilio. La pantalla muestra información de los tiempos hábiles de entrega e incluye una imágen ilustrativa del lado derecho. *Enlace a la vista: Envío.*
- Pago - Tarjeta: Sección donde se realiza el pago mediante la integración de Mercado Pago. El usuario ingresa el nombre del titular, número de tarjeta, mes, año y código de seguridad. A la derecha se mantiene una imagen ilustrativa. *Enlace a la vista: Pago - Tarjeta.*
- Verificación de pago: Pantalla de éxito que le informa al usuario que su compra está siendo procesada, incluyendo un mensaje de agradecimiento y un botón central para volver rápidamente al inicio. *Enlace a la vista: Verificación de pago.*

**Vistas del lado del administrador.**

- Dashboard: Pantalla en la que el administrador visualiza métricas e información clave sobre el rendimiento del e-commerce. Incluye un perfil a la izquierda, un menú lateral permanente de navegación y un resumen de estadísticas que detalla los usuarios, actividad y un gráfico con el “Top 5 mates más vendidos”. *Enlace a la vista: Dashboard.*
- Sección productos: Página de entrada para la gestión del catálogo que ofrece de manera directa y minimalista el acceso para visualizar la lista completa de los artículos registrados o ingresar al formulario de agregado. *Enlace a la vista: Sección productos.*
- Lista de productos: Vista centralizada en formato de tabla que detalla el catálogo. Cada fila muestra el nombre, precio, descripción y color de un producto, e incorpora botones de acción para editar o eliminar cada ítem. *Enlace a la vista: Lista de productos.*
- Modificar producto: Formulario al que se accede cuando se quiere editar. Permite actualizar datos como el nombre, precio, descripción, color, tipo de

producto y la gestión de sus imágenes. Finaliza con un botón para guardar los cambios. *Enlace a la vista: Modificar producto.*

- Clientes: Vista en formato lista que agrupa a todos los clientes registrados. Permite acceder a los datos relevantes de cada usuario y monitorear el comportamiento de compra dentro de la tienda. *Enlace a la vista: Clientes.*

Todos los diseños y wireframes detallados en esta sección pueden visualizarse directamente en el archivo de Figma del proyecto: Figma Wireframes Mate Único

## MOCKUPS.

Se desarrollaron prototipos de alta fidelidad en Figma para definir la identidad visual y el flujo de navegación de la plataforma. Estos mockups permiten interactuar con la interfaz del sistema, integrando la estructura de los wireframes previos con la estética final. El acceso al diseño completo se encuentra en el siguiente enlace: Figma Mockups Mate Único

## CASOS DE USO Y ESPECIFICACIONES.

El presente diagrama de casos de uso representa el modelo de datos para una tienda online especializada en la venta de mates, bombillas y portatermos. De esta manera representa las interacciones que puede hacer un cliente o administrador dentro de la web.

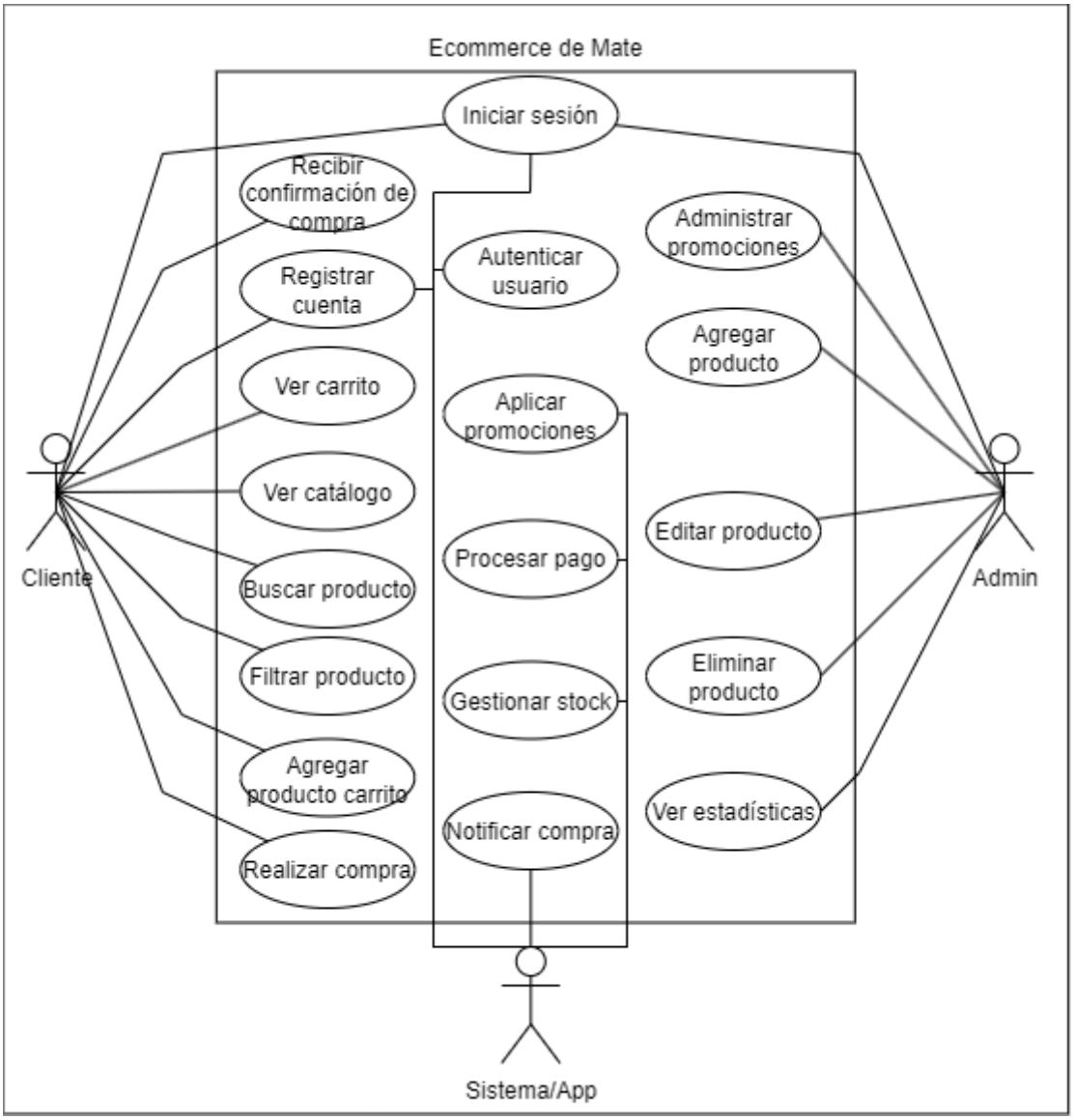



*Figura 1. Diagrama de Casos de Uso del E-Commerce "Mate Único".*

**Especificaciones de Casos de Uso.**

**Casos de Uso para Cliente:**

**Registro:**

- Actor: Cliente
- Descripción: Permite al cliente crear una cuenta para poder comprar en el ecommerce.
- Precondición: El cliente no debe tener una cuenta registrada a ese correo.
- Postcondición: La cuenta queda creada y lista para iniciar sesión.
- Secuencia principal:
    1. El cliente selecciona en “Registrarse”.
    2. El sistema solicita los datos para registro.
    3. El cliente completa y envía.
    4. El sistema valida los datos y crea la cuenta.
    5. El sistema envía correo de confirmación.
- Flujo alternativo:
    a. Si el correo ya existe, el sistema muestra error.}

**Inicio de sesión:**

- Actor: Cliente
- Descripción: Permite al cliente acceder a la plataforma ingresando con sus datos.
- Precondición: El cliente no debe haber iniciado sesión en la página y debe estar registrado.
- Postcondición: El cliente queda logueado en el sistema.
- Secuencia principal:
    1. El cliente coloca los datos correctos (email, contraseña) en el formulario.
    2. El sistema valida las credenciales.
    3. Si los datos son correctos se inicia sesión en la web y se puede navegar libremente.

- Flujo alternativo:
    a. Si no tiene una cuenta creada, se le creará una con su correo electrónico.

**Navegar productos:**

- Actor: Cliente
- Descripción: Consultar catálogo de productos disponibles para su compra.
- Precondición: Estar en la sección correcta de la web.
- Postcondición: El cliente ve los productos.
- Secuencia principal:
    1. El cliente accede a la sección de catálogo.
    2. El sistema muestra todos los datos necesarios sobre los productos.
    3. El cliente puede buscar o aplicar algún filtro de su preferencia.
- Flujo alternativo
    a. Ninguno.

**Agregar productos al carrito:**

- Actor: Cliente
- Descripción: Seleccionar productos para una posible compra.
- Precondición: Haber elegido un producto y tener stock del mismo.
- Postcondición: El producto queda en el carrito listo para su compra.
- Secuencia principal:
    1. El cliente selecciona un producto.
    2. El sistema muestra todos los detalles y datos disponibles del mismo.

3. Si el cliente está interesado clickea en agregar al carrito/comprar.
4. El sistema verifica que hay stock disponible.
5. De ser así, el sistema confirma y agrega el/los producto/s al carrito.
6. El sistema actualiza el nuevo stock disponible y la información del carrito.

- Flujo alternativo:
    a. Si no hay stock suficiente, el sistema muestra un mensaje de error.

**Finalizar compra:**

- Actor: Cliente
- Descripción: Pagar y adquirir el producto del carrito.
- Precondición: Tener como mínimo un producto en el carrito y estar logueado.
- Postcondición: La compra se registra y se envía el comprobante al cliente.
- Secuencia principal:
    1. El cliente revisa su carrito y verifica la compra.
    2. El sistema valida stock y aplica posibles promociones/cupones.
    3. El cliente selecciona el proveedor de envío.
    4. El cliente selecciona el método de pago.
    5. El sistema redirige hacia la sección de pago.
    6. El cliente completa los datos y realiza el pago.
    7. El sistema confirma el pago y genera la orden de compra.
    8. El sistema envía por mail el comprobante de la compra.

- Flujo alternativo:
    a. Si no hay stock, el sistema notifica.
    b. Si el pago es rechazado, el sistema muestra error y permite reintentar.
    c. Si el cliente cancela, la compra se interrumpe y el carrito se conserva.

**Comentario/Puntuación de producto:**

- Actor: Cliente
- Descripción: Hacer una reseña del producto
- Precondición: El cliente debe haber comprado el producto.
- Postcondición: La valoración queda registrada y disponible para su vista.
- Secuencia principal:
    1. El cliente accede a un producto.
    2. Selecciona la opción de comentar/puntuar.
    3. Escribe un comentario o asigna cierta cantidad de estrellas.
    4. El sistema guarda la información y se publica la valoración.
- Flujo alternativo:
    a. Ninguno.

**Modificar datos:**

- Actor: Cliente
- Descripción: Editar dirección de domicilio.
- Precondición: Estar logueado
- Postcondición: Dirección actualizada para compras futuras.
- Secuencia principal:
    1. El cliente ingresa a su perfil.

2. Se ingresa a la sección de modificación.
3. Edita el campo disponible (dirección).
4. Guarda cambios
5. El sistema actualiza la nueva información.

- Flujo alternativo:
    a. Si falta algún dato obligatorio, el sistema no guarda cambios.

**Casos de Uso para Administrador:**

**Iniciar sesión:**

- Actor: Admin
- Descripción: Ingresar a la web con el usuario dado por los desarrolladores.
- Precondición: Tener una cuenta de administrador funcional.
- Postcondición: Ingreso al dashboard.
- Secuencia principal:
    1. El admin ingresa al apartado de inicio de sesión.
    2. Completa los datos en los campos solicitados.
    3. El sistema verifica los datos, confirma que son datos de administrador y envía la petición de ingreso.
    4. Si los datos son correctos, muestra el panel de administración.
- Flujo alternativo:
    a. Si los datos son inválidos, se rechaza el acceso.

**Agregar productos:**

- Actor: Admin
- Descripción: Agregar productos para la venta
- Precondición: Estar logueado como admin.

- Postcondición: Se crea la publicación del producto y queda listo para su venta.
- Secuencia principal:
    1. El admin ingresa a la sección de “Agregar producto”.
    2. Completa los datos solicitados (nombre, precio, descripción, materiales, etc.)
- Flujo alternativo:
    a. Si faltan datos, no se completa la publicación.

**Eliminar productos:**

- Actor: Admin
- Descripción: Eliminar producto listado.
- Precondición: Estar logueado como admin.
- Postcondición:El producto queda eliminado del catálogo.
- Secuencia principal:
    1. El admin selecciona el producto a eliminar.
    2. Confirma la solicitud.
    3. El sistema valida y elimina el producto del catálogo.
- Flujo alternativo:
    a. Si el producto tiene ventas pendientes, no puede eliminarse.

**Modificar producto:**

- Actor: Admin
- Descripción: Modificar datos de un producto.
- Precondición: Estar logueado como admin
- Postcondición: El producto queda listo con las modificaciones hechas.
- Secuencia principal:

1. El admin selecciona el producto de la lista.
2. Modifica los datos del mismo (nombre, descripción, stock, imágenes, etc).
3. El sistema guarda los datos y se actualiza la información.

- Flujo alternativo:
    a. Si los cambios no son válidos, el sistema no actualiza el producto.

**Visualizar estadísticas:**

- Actor: Admin
- Descripción: Ver las estadísticas del e-commerce.
- Precondición: Estar logueado como admin,
- Postcondición: El admin conoce las estadísticas del e-commerce.
- Secuencia principal:
    1. El administrador ingresa al panel o sección de reportes.
    2. De ser necesario selecciona filtros para una mejor visualización (fecha, producto).
    3. El sistema muestra todos los datos sobre estadísticas (productos top, facturación, ventas, clientes, etc).
- Flujo alternativo:
    a. Ninguno.

**Administrar promociones:**

- Actor: Admin
- Descripción: Agregar/eliminar/editar promociones.
- Precondición: Estar logueado como admin
- Postcondición: Se modifican las promociones del catálogo.
- Secuencia principal:

1. El admin ingresa a “promociones”.
2. Crea, edita o elimina promociones (2x1, cupones, hot sale, etc).
3. Guarda cambios.
4. El sistema aplica las promociones y se actualiza el catálogo.

- Flujo alternativo:
    a. Si el formato de promoción es incorrecto, se rechazan los cambios.

**Casos de Uso para Sistema/App:**

**Autenticar usuario:**

- Actor: Sistema
- Descripción: Validar credenciales de cliente/admin.
- Precondición: El usuario debe haber enviado las credenciales.
- Postcondición: El usuario se registra/loguea o se muestra mensaje de error.
- Secuencia principal:
    1. El cliente/admin envía usuario y contraseña.
    2. El sistema verifica en la BD.
    3. Si los datos coinciden genera una sesión activa.
    4. Si son incorrectos, rechaza acceso.
- Flujo alternativo:
    a. Ninguno.

**Aplicar promociones:**

- Actor: Sistema
- Descripción: Actualizar catálogo de productos con las promos definidas por el administrador.

- Precondición: El admin debe haber guardado los cambios.
- Postcondición: Se actualiza el catálogo con las promos aplicadas.
- Secuencia principal:
    1. Recibe los cambios por el administrador.
    2. Valida que las promociones tengan el formato correspondiente.
    3. Aplica la promoción sobre los productos elegidos.
    4. Actualizar el catálogo disponible para los clientes.
- Flujo alternativo:
    a. Si la promo es inválida no se aplica.

**Enviar confirmación de compra:**

- Actor: Sistema
- Descripción: Notificar al cliente la confirmación de su compra.
- Precondición: El cliente debe haber finalizado el proceso de compra.
- Postcondición: El cliente recibe la confirmación de compra junto con los detalles de la misma.
- Secuencia principal:
    1. Recibe la orden de compra del cliente.
    2. Verifica que el pago se procesó correctamente.
    3. Genera un comprobante de compra.
    4. Envía la confirmación al correo electrónico del clietne.
- Flujo alternativo:
    a. Si el pago falla, no se envía la confirmación.

**Actualizar estado de inventario:**

- Actor: Sistema
- Descripción: Mantener actualizado el stock de productos.

- Precondición: Se confirma una compra por parte del cliente.
- Postcondición: El inventario queda actualizado y el admin recibe la alerta correspondiente.
- Secuencia principal:
    1. Recibe la info de una venta.
    2. Descuenta automáticamente la cantidad comprada del inventario.
    3. Verifica si hay pocas unidades de algún producto.
    4. Si el stock es bajo, genera una alerta para el admin.
- Flujo alternativo:
    a. Ninguno.

**Generar estadísticas de ventas:**

- Actor: Sistema
- Descripción: Proveer información al admin. sobre el negocio.
- Precondición: Deben existir datos de ventas registradas.
- Postcondición: El admin accede a reportes con las estadísticas actualizadas.
- Secuencia principal:
    1. Registra cada compra.
    2. Agrupa los datos de venta por filtros (fecha, producto, monto, etc.).
    3. Procesa la información en dashboards.
    4. Muestra los resultados al admin cuando lo solicita.
- Flujo alternativo:
    a. Ninguno.

**Procesar pago:**

- Actor: Sistema
- Descripción: Ejecutar validación y confirmación de pago.
- Precondición: Carrito confirmado y datos de pago válidos.
- Postcondición: Compra registrada (o rechazada).
- Secuencia principal:
    1. El cliente confirma el carrito.
    2. El sistema conecta con el medio de pago.
    3. Recibe respuesta de aprobado/rechazado.
    4. Si se aprueba, registra compra y genera comprobante.
- Flujo alternativo:
    a. Si el pago es rechazado, la compra no se registra.

## PLANIFICACIÓN Y METODOLOGÍA.

### Metodología de Trabajo.

Para el desarrollo del e-commerce “Mate Único”, se implementó un enfoque de desarrollo híbrido combinando elementos predictivos y adaptativos. Este marco metodológico permite establecer una hoja de ruta clara, manteniendo la flexibilidad necesaria en las etapas de implementación y prueba. El proceso se divide en dos fases fundamentales:

- Etapa predictiva: planificación inicial, definición de requerimientos, diseño de base de datos, diagramas, mockups y arquitectura.
- Etapa adaptativa: desarrollo incremental del sistema en iteraciones cortas, con revisiones frecuentes del producto y ajustes según los resultados obtenidos.

El desarrollo del sistema se llevará a cabo en distintas fases secuenciales y complementarias. Durante la etapa de implementación y prueba, el trabajo se va a organizar mediante “sprints” permitiendo una entrega progresiva, sin perder continuidad del objetivo y el producto final.

Cada sprint incluirá tareas de codificación, revisión, documentación y prueba, de manera que al finalizar el ciclo se obtiene una versión funcional y continua del sistema.

**Planificación.**

El proyecto contempla una duración total estimada de 16 semanas para su implementación. Las actividades de la etapa predictiva (relevamiento, requerimientos, diseño y prototipado) se desarrollaron entre agosto y octubre de 2025.

El ciclo de desarrollo técnico se proyecta desde el mes de noviembre de 2025 hasta junio de 2026, contemplando una interrupción de actividades durante el mes de enero de 2026 debido al receso académico

Para organizar la ejecución y asegurar el cumplimiento progresivo de los objetivos, el esfuerzo se ha estructurado en las siguientes seis fases principales:

- **Fase 1:** Diseño técnico (Noviembre - Diciembre 2025): Etapa inicial que comprende la configuración del entorno de desarrollo, la definición de la arquitectura base del sistema y el diseño de los endpoints.
- **Fase 2:** Desarrollo Back-End (Febrero 2026): Período enfocado en la implementación de la API RESTful, la conexión formal con la base de datos PostgreSQL y la ejecución de las primeras pruebas de servidor.
- **Fase 3:** Fin Back - Comienzo Front (Marzo 2026): Fase de transición que abarca la creación de las vistas gráficas de la interfaz web y el inicio de la integración de estas con los servicios de la API.
- **Fase 4:** Desarrollo Front-End (Abril 2026): Consiste en el cierre del desarrollo puro de la interfaz de usuario, culminando la maquetación y asegurando la correcta interacción del cliente con el sistema.

- **Fase 5:** Integración y pruebas (Mayo 2026): Abarca la validación funcional integral del flujo de compra, la corrección de errores detectados y la optimización del rendimiento del sistema.
- **Fase 6**: Documentación final (Junio 2026): Cierre formal del proyecto que incluye la elaboración de todos los manuales y documentos técnicos correspondientes, culminando con la presentación final y demostración del sistema.

**Herramientas de Gestión.**

Para asegurar una coordinación eficiente y mantener la trazabilidad integral del ciclo de vida del sistema, el equipo empleará diversas herramientas tecnológicas y metodológicas:

- Control de versiones: El desarrollo, seguimiento y resguardo de todo el código fuente se administrará a través de un sistema de control de versiones utilizando Git.
- Diseño y Prototipado: La elaboración de wireframes y el diseño de mockups se centralizará de forma colaborativa a través de la plataforma Figma.
- Seguimiento de Tareas y Comunicación: La gestión del proyecto se apoyará en técnicas y metodologías ágiles, organizando el esfuerzo y el progreso mediante sprints. Como estrategia definida para la coordinación del equipo, se mantendrá un control y seguimiento constante a través de múltiples vías de comunicación digitales, tales como Discord.

## REPOSITORIO DEL PROYECTO.

El código del proyecto se encuentra centralizado en un repositorio de GitHub.

En este espacio se aloja tanto la lógica del servidor como el desarrollo de la

interfaz de usuario y los scripts de la base de datos. Por razones de seguridad, se omitieron del repositorio público los archivos de configuración que contienen información sensible, tales como las variables de entorno y las credenciales de las API integradas en el sistema.
Enlace al repositorio: https://github.com/SantinoDacuy/MATE-UNICO-Taller

## PRESENTACIÓN TÉCNICA DEL PROYECTO.

El sistema está estructurado bajo una arquitectura desacoplada y dividida en capas independientes. La capa de presentación orientada al cliente está construida utilizando React junto con Vite, lo cual asegura una navegación sumamente rápida e interactiva. El diseño visual se apoya en lenguajes web nativos como HTML, CSS y JavaScript, garantizando una interfaz moderna y adaptable.
La lógica de negocio y el servidor operan sobre un entorno Node.js empleando el marco de trabajo Express, actuando como el encargado de procesar las validaciones de inventario y gestionar el flujo de compras. La persistencia de los datos queda resguardada por el motor de base de datos relacional PostgreSQL. Todo este ecosistema se complementa con la integración de servicios externos fundamentales, incorporando protocolo de autenticación de Google para un acceso seguro y la pasarela de Mercado Pago para procesar las transacciones financieras.

### Tecnologías Implementadas: Funcionalidad y Justificación.

El proyecto emplea un stack tecnológico robusto basado en una arquitectura de servicios desacoplados, diseñada para ofrecer una experiencia de e-commerce fluida y escalable. La plataforma integra un frontend moderno de React, un CMS para la gestión de productos y un backend dedicado para la lógica

de negocio y autenticación, asegurando una separación clara de responsabilidades.

**Frontend**

- **React (Vite):**Elegimos React para crear una interfaz dinámica y rápida mediante el uso de componentes reutilizables. Lo combinamos con Vite ya que acelera el tiempo de carga y desarrollo en comparación con otras herramientas. Fue nuestra opción principal por su gran rendimiento.
- **JavaScript (JSX):** Usamos JSX para escribir el código visual junto con la lógica de programación en un mismo archivo, lo que hace el código más claro. Esto nos permitió mostrar los productos y actualizar el carrito de compras al instante sin que el usuario tenga que recargar la página.
- **CSS3:** Decidimos trabajar con CSS puro para tener un control total sobre el diseño y evitar librerías externas. Esta selección prioriza la velocidad de carga y una estética limpia y personalizada.

**Backend**

- **Node.js con Express:** Usamos Node.js para el servidor porque permite manejar muchas conexiones al mismo tiempo de manera eficiente. Express nos facilitó la creación de las rutas de la API que conectan los datos con la interfaz de forma sencilla.
- **Strapi (Headless CMS):** Seleccionamos Strapi para gestionar los productos y las categorías sin tener que programar un panel de administración desde cero. Nos permitió ahorrar mucho tiempo de desarrollo y tener una base de datos lista para ser consumida por el frontend. Es una herramienta muy flexible que nos da autonomía para manejar el contenido del e-commerce.
- **PostgreSQL:** Elegimos este motor de base de datos relacional porque garantiza que la información de las ventas y los usuarios sea siempre

exacta y segura. Es muy robusto y está preparado para manejar grandes volúmenes de datos sin errores de integridad. Su integración con Strapi fue inmediata, lo que nos dio una base sólida para el proyecto.

- **Mercado Pago API (Checkout Pro):** Implementamos Mercado Pago para que la tienda pueda procesar cobros reales de forma segura y sencilla. Elegimos Checkout Pro porque ofrece confianza a los clientes y ya incluye todos los medios de pago locales. Además, nos permite recibir avisos automáticos cuando un pago es aprobado para actualizar el pedido.
- **ngrok:** Esta herramienta fue clave en el desarrollo para poder conectar nuestro servidor local con internet de forma temporal. La usamos principalmente para probar que los avisos de pago de Mercado Pago llegaran correctamente a nuestra base de datos. Sin ngrok, no habríamos podido testear el flujo completo de compra desde nuestra computadora.

**Descripción de la Arquitectura.**

El proyecto se basa en una arquitectura de servicios desacoplados, donde el frontend y el backend funcionan de manera independiente y se comunican a través de una API REST. Esta estructura permite que cada parte del sistema pueda escalar o modificarse sin afectar directamente a la otra, mejorando la organización del código y facilitando el mantenimiento a largo plazo. Los componentes principales son:

- **Capa de Presentación:** Desarrollada con React, se encarga de gestionar la interfaz de usuario de forma dinámica. Al ser una Single Page Application (SPA), las transiciones entre páginas son instantáneas, ya que solo se intercambian datos en formato JSON con el servidor en lugar de recargar archivos HTML completos.
- **Capa de Negocio y Contenidos:** Centralizada en Strapi, que funciona como un Headless CMS sobre un entorno de Node.js. Esta capa gestiona

la lógica de los productos, el control de stock y el almacenamiento de la información en una base de datos PostgreSQL. Strapi expone los endpoints necesarios para que el frontend pueda consultar el catálogo de mates y procesar las órdenes.

- **Capa de Integración de Pagos:** Se utiliza la API de Mercado Pago como un servicio externo para procesar las transacciones de forma segura. El flujo se completa mediante el uso de webhooks que notifican al backend cuando un pago es exitoso, permitiendo la actualización del estado del pedido.
- **Infraestructura de Desarrollo:** Durante la etapa de pruebas, se implementó ngrok para crear un túnel seguro que permite al servidor de Mercado Pago comunicarse con nuestro entorno local. Esto fue vital para simular el comportamiento de la tienda en un entorno real de producción.

**Beneficios de la arquitectura**

- **Escalabilidad y Flexibilidad:** Al estar el frontend y el backend desacoplados, el sistema puede crecer de forma independiente. Si en el futuro se decide lanzar una aplicación móvil, esta puede consumir la misma API de Strapi sin necesidad de modificar la lógica del servidor o la base de datos PostgreSQL.
- **Seguridad y Robustez en la Gestión de Datos:** El uso de un Headless CMS como Strapi permite centralizar la administración de contenidos en un entorno seguro. Esto, sumado a la integridad referencial de PostgreSQL, asegura que la información de stock, precios y pedidos sea consistente y esté protegida contra errores de concurrencia.
- **Mantenimiento Simplificado:** La separación de responsabilidades permite que los errores se localicen y solucionen más rápido. Si

surge un problema visual, se trabaja exclusivamente sobre el código de React; si el problema es de datos o pagos, se interviene sobre el backend o la integración con Mercado Pago, sin afectar la estabilidad de las otras capas.

- **Integración de Servicios de Terceros:** La arquitectura modular facilitó la implementación de servicios externos como Mercado Pago para el checkout y ngrok para las pruebas de integración. Estas herramientas se conectan de forma natural al flujo del backend, permitiendo que el proyecto cuente con funcionalidades de nivel profesional sin aumentar la complejidad del código base.

**Flujo de comunicación.**

El intercambio de información en la plataforma sigue un ciclo de vida de petición-respuesta orientado a eventos, estructurado en los siguientes pasos:

- **Petición del Cliente:** El usuario interactúa con la interfaz en React. Cuando se solicita el catálogo o se agrega un producto, el frontend realiza una petición asíncrona hacia el backend.
- **Gestión de Datos:** Strapi, corriendo sobre Node.js, recibe la solicitud y consulta la base de datos PostgreSQL. El CMS procesa la lógica de negocio y devuelve la información.
- **Procesamiento de Pago:** Al confirmar la compra, el sistema redirige al usuario al Checkout Pro de Mercado Pago. En este punto, la comunicación sale de nuestro servidor hacia la API externa del proveedor de pagos.
- **Notificación de Estado:** Una vez que el usuario paga, Mercado Pago envía una notificación automática a un endpoint de nuestro backend. Durante el desarrollo, ngrok actúa como el puente necesario para que esta señal externa llegue a nuestra computadora local.

- **Actualización y Confirmación:** El backend recibe la confirmación, actualiza el estado del pedido en PostgreSQL y envía una señal de éxito al frontend, que actualiza la interfaz del usuario sin necesidad de recargar la página.

## MODELO DE DATOS DER.

A continuación se presenta el Diagrama Entidad-Relación (DER) que detalla la estructura lógica de la base de datos en PostgreSQL. En este esquema se visualizan las entidades principales del e-commerce y las relaciones que garantizan la integridad de los datos.

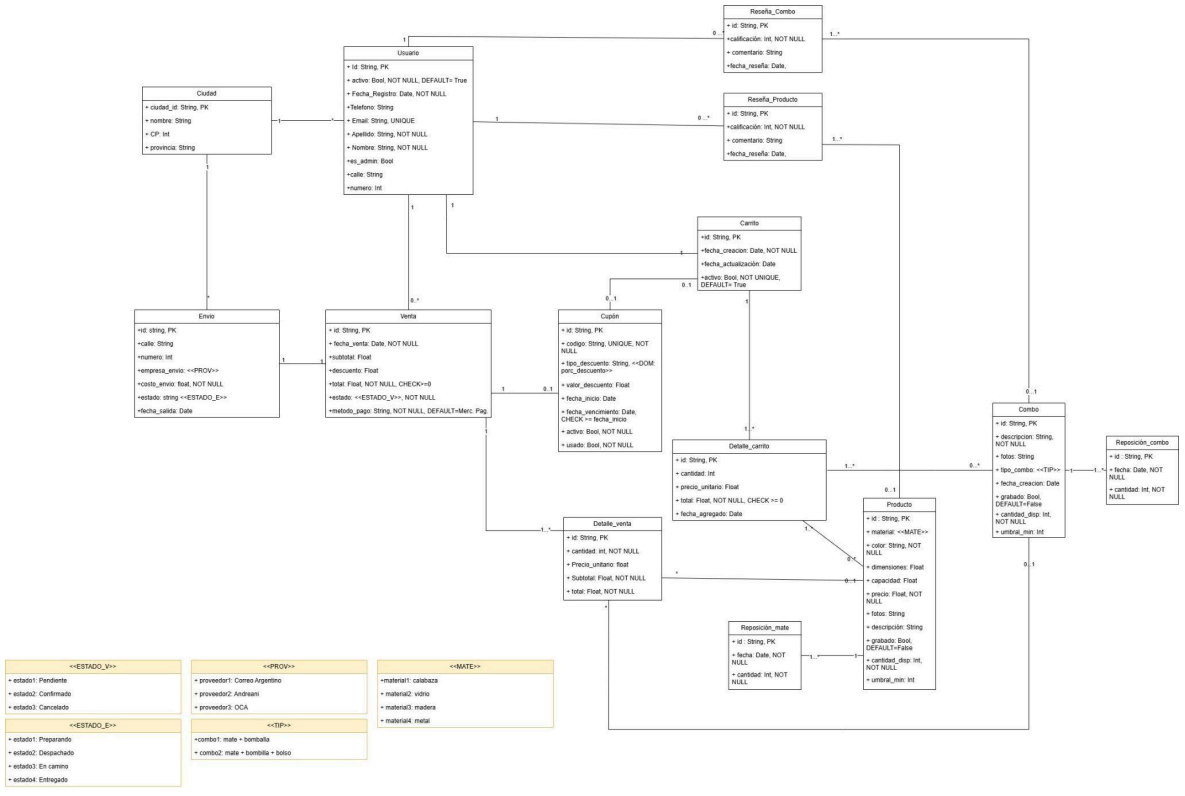



*Figura 2. Diagrama Entidad - Relación.*

Para una visualización más detallada se puede acceder al diagrama a través del siguiente enlace: Diagrama Entidad-Relación.

**Diccionario de Datos.**

Usuario

| Atributo | Dominio | Valor nulo | Valor por defecto | Rango válido | Clave | Descripción |
|---|---|---|---|---|---|---|
| **id** | String | No | - | - | PK | Identificador único del usuario |
| **Nombre** | String | No | - | - | - | Nombre del usuario |
| **Apellido** | String | No | - | - | - | Apellido del usuario |
| **Email** | String | No | - | Formato email | UNIQUE | Correo electrónico del usuario |
| **Telefono** | String | Si | NULL | - | - | Número de teléfono de contacto |
| **Direccion** | String | Si | NULL | - | - | Dirección física del usuario |
| **es_admin** | Bool | No | False | {True, False} | - | Indica si el usuario es administrador |

| | | | | | | |
|---|---|---|---|---|---|---|
| **activo** | Bool | No | True | {True, False} | - | Indica si la cuenta está activa |
| **Fecha_Registro** | Date | No | - | - | - | Fecha de registro en el sistema |

Carrito

| Atributo | Dominio | Valor nulo | Valor por defecto | Rango válido | Clave | Descripción |
|---|---|---|---|---|---|---|
| **id** | String | No | - | - | PK | Identificador único del carrito |
| **fecha_creacion** | Date | No | - | - | - | Fecha de creación del carrito |
| **fecha_actualización** | Date | Si | NULL | - | - | Última fecha de modificación |
| **activo** | Bool | No | True | {True, False} | - | Indica si el carrito está activo |

Producto

| Atributo | Dominio | Valor nulo | Valor por defecto | Rango válido | Clave | Descripción |
|---|---|---|---|---|---|---|
| **id** | String | No | - | - | PK | Identificador único del |

| | | | | | | |
|---|---|---|---|---|---|---|
| | | | | | | producto |
| **descripción** | String | No | - | - | - | Descripción detallada del producto |
| **material** | String | No | - | {Calabaza, Vidrio, Madera, Metal} | - | Material del producto |
| **color** | String | No | - | - | - | Color del producto |
| **dimensiones** | Float | Si | NULL | >0 | - | Dimensiones en cm |
| **capacidad** | Float | Si | NULL | >0 | - | Capacidad en ml |
| **precio** | Float | No | - | >0 | - | Precio unitario del producto |
| **fotos** | String | No | - | - | - | URLs de las imágenes del producto |
| **grabado** | Bool | Si | False | {True, False} | - | Indica si permite grabado personalizado |
| **cantidad_disp** | Int | No | 0 | >=0 | - | Stock disponible del producto |

| | | | | | | |
|---|---|---|---|---|---|---|
| **umbral_min** | Int | No | 2 | >=0 | - | Stock mínimo para alerta |

Combo

| Atributo | Dominio | Valor nulo | Valor por defecto | Rango válido | Clave | Descripción |
|---|---|---|---|---|---|---|
| **Id** | String | No | - | - | PK | Identificador único del combo |
| **descripcion** | String | No | - | - | - | Descripción del combo |
| **tipo_combo** | String | No | - | {Mate+Bombilla, Mate+Bombilla+Bolso} | - | Tipo de combinación del combo |
| **fotos** | String | No | - | - | - | URLs de imágenes del combo |
| **grabado** | Bool | Si | False | {True, False} | - | Indica si el combo permite grabado |
| **cantidad_disp** | Int | No | 0 | >=0 | - | Stock disponible del combo |
| **umbral_min** | Int | No | 2 | >=0 | - | Stock mínimo |

| | | | | | | |
|---|---|---|---|---|---|---|
| | | | | | | para alerta |
| **fecha_creacion** | Date | No | - | - | - | Fecha de creación del combo |

Detalle_carrito

| Atributo | Dominio | Valor nulo | Valor por defecto | Rango válido | Clave | Descripción |
|---|---|---|---|---|---|---|
| **id** | String | No | - | - | PK | Identificador único del detalle |
| **cantidad** | Int | No | - | > 0 | - | Cantidad del ítem |
| **precio_unitario** | Float | No | - | > 0 | - | Precio unitario al momento de agregar |
| **total** | Float | No | - | >= 0 | - | Total de la venta |
| **fecha_agregado** | Date | No | - | - | - | Fecha en que se agregó al carrito |

Venta

| Atributo | Dominio | Valor nulo | Valor por defecto | Rango válido | Clave | Descripción |
|---|---|---|---|---|---|---|
| **id** | String | No | - | - | PK | Identificador único de la |

| | | | | | | |
|---|---|---|---|---|---|---|
| | | | | | | venta |
| **fecha_venta** | Date | No | - | - | - | Fecha de la venta |
| **subtotal** | Float | No | - | >=0 | - | Subtotal antes de descuentos |
| **descuento** | Float | No | 0 | >=0 | - | Monto de descuento aplicado |
| **total** | Float | No | - | >=0 | - | Total final de la venta |
| **estado** | String | No | ‘Pendiente’ | {Pendiente, Confirmado, Cancelado} | - | Estado de la venta |
| **metodo_pago** | String | No | ‘Mercado Pago’ | - | - | Método de pago usado |

Detalle_venta

| Atributo | Dominio | Valor nulo | Valor por defecto | Rango válido | Clave | Descripción |
|---|---|---|---|---|---|---|
| **id** | String | No | - | - | PK | Identificador único del detalle |
| **cantidad** | Int | No | - | >0 | - | Cantidad vendida |
| **precio_unitario** | Float | No | - | >0 | - | Precio unitario al momento de la venta |
| **subtotal** | Float | No | - | >=0 | - | Subtotal (cantidad * precio_unitario) |
| **total** | Float | No | - | >=0 | - | Precio final |

Envío

| Atributo | Dominio | Valor nulo | Valor por defecto | Rango válido | Clave | Descripción |
|---|---|---|---|---|---|---|
| **id** | String | No | - | - | PK | Identificad |

| | | | | | | |
|---|---|---|---|---|---|---|
| | | | | | | or único del envío |
| **direccion** | String | No | - | - | - | Dirección de entrega |
| **empresa_envio** | String | No | NULL | {Correo Argentino, Andreani, OCA} | - | Empresa de envío seleccionada |
| **costo_envio** | Float | No | - | >=0 | - | Costo del envío |
| **estado** | String | No | 'Preparando' | {Preparando, Despachado, En camino, Entregado} | - | Estado del envío |
| **fecha_salida** | Date | No | NULL | - | - | Fecha de despacho del pedido |

Cupón

| **Atributo** | **Dominio** | **Valor nulo** | **Valor por defecto** | **Rango válido** | **Clave** | **Descripción** |
|---|---|---|---|---|---|---|

| | | | | | | |
|---|---|---|---|---|---|---|
| **id** | String | No | - | - | PK | Identificador único del cupón |
| **codigo** | String | No | - | - | UNIQUE | Código del cupón |
| **tipo_descuento** | String | No | - | {porcentaje, monto_fijo} | - | Tipo de descuento del cupón |
| **valor_descuento** | Float | No | - | >0 | - | Valor del descuento |
| **fecha_inicio** | Date | No | - | - | - | Fecha desde la cual es válido |
| **fecha_vencimiento** | Date | No | - | >= fecha_inicio | - | Fecha hasta la cual es válido |
| **activo** | Bool | No | True | {True, False} | - | Indica si el cupón está activo |
| **usado** | Bool | No | False | {True, False} | - | Muestra si el cupón |

| | | | | | | |
|---|---|---|---|---|---|---|
| | | | | | | fue usado |

Reseña_Combo

| **Atributo** | **Dominio** | **NULL** | **Valor por defecto** | **Rango Válido** | **Clave** | **Descripción** |
|---|---|---|---|---|---|---|
| **id** | String | No | - | - | PK | Identificador de la reseña del combo |
| **calificación** | Int | No | - | 1 <= x <= 5 | - | Calificación de 1 a 5 estrellas |
| **comentario** | String | Si | NULL | - | - | Comentario opcional del usuario |
| **fecha_reseña** | Date | No | - | - | - | Fecha de publicación de la reseña |

Reseña_Producto

| **Atributo** | **Dominio** | **NULL** | **Valor por defecto** | **Rango Válido** | **Clave** | **Descripción** |
|---|---|---|---|---|---|---|

| | | | | | | |
|---|---|---|---|---|---|---|
| **id** | String | No | - | - | PK | Identificador de la reseña del producto |
| **calificación** | Int | No | - | 1 <= x <= 5 | - | Calificación de 1 a 5 estrellas |
| **comentario** | String | Si | NULL | - | - | Comentario opcional del usuario |
| **fecha_reseña** | Date | No | - | - | - | Fecha de publicación de la reseña |

Reposicion_mate

| **Atributo** | **Dominio** | **NULL** | **Valor por defecto** | **Rango Válido** | **Clave** | **Descripción** |
|---|---|---|---|---|---|---|
| **id** | String | No | - | - | PK | Identificador de reposición |
| **fecha** | Date | No | - | - | - | Fecha de reposición |
| **cantidad** | Int | No | - | - | - | Cantidad |

| | | | | | | |
|---|---|---|---|---|---|---|
| | | | | | | repuesta |

Reposicion_combo

| **Atributo** | **Dominio** | **NULL** | **Valor por defecto** | **Rango Válido** | **Clave** | **Descripción** |
|---|---|---|---|---|---|---|
| **id** | String | No | - | - | PK | Identificador de reposición |
| **fecha** | Date | No | - | - | - | Fecha de reposición |
| **cantidad** | Int | No | - | - | - | Cantidad repuesta |

## ESTRUCTURA DE CARPETAS DEL PROYECTO

**Frontend:**

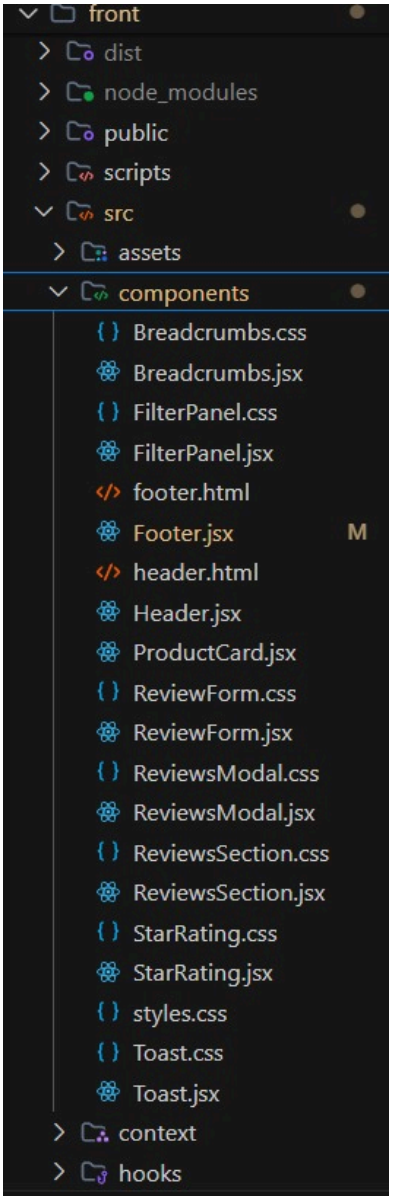



*Figura 3. Arquitectura del Frontend (components).*

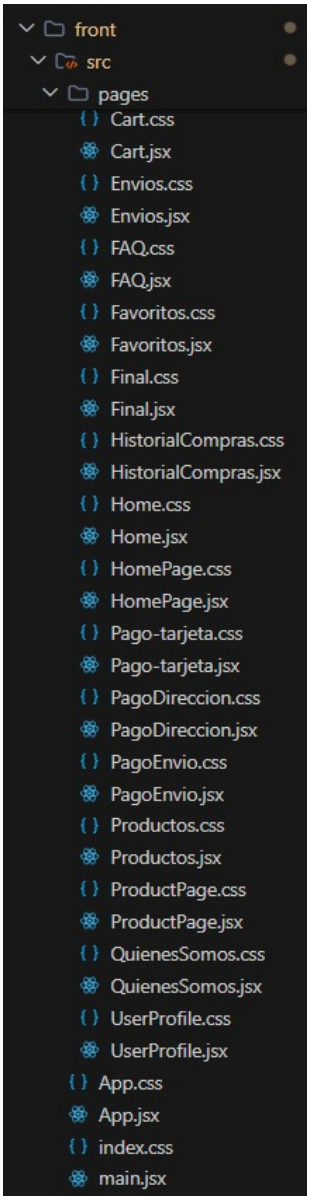



*Figura 4. Arquitectura del Frontend (pages - Primera parte).*

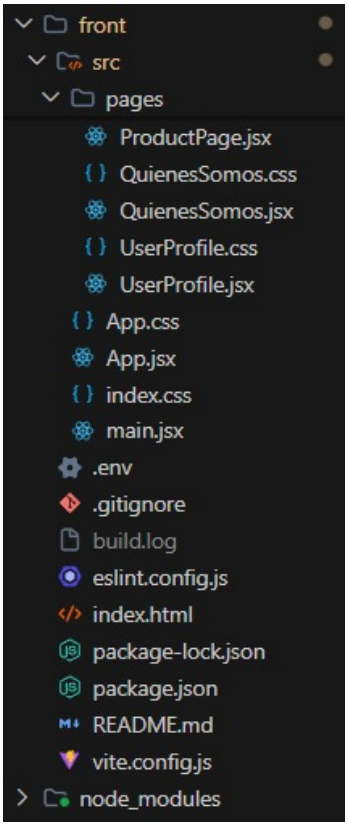



*Figura 5. Arquitectura del Frontend (pages - Segunda parte).*

La arquitectura del frontend está organizada de forma modular bajo la carpeta src, separando las piezas visuales reutilizables en /components de las vistas completas del sitio en /pages. Esta estructura, potenciada por Vite, utiliza /hooks y /context para gestionar la lógica y el estado global de la tienda sin mezclarla con el diseño, logrando un código escalable y fácil de mantener.

**Backend:**

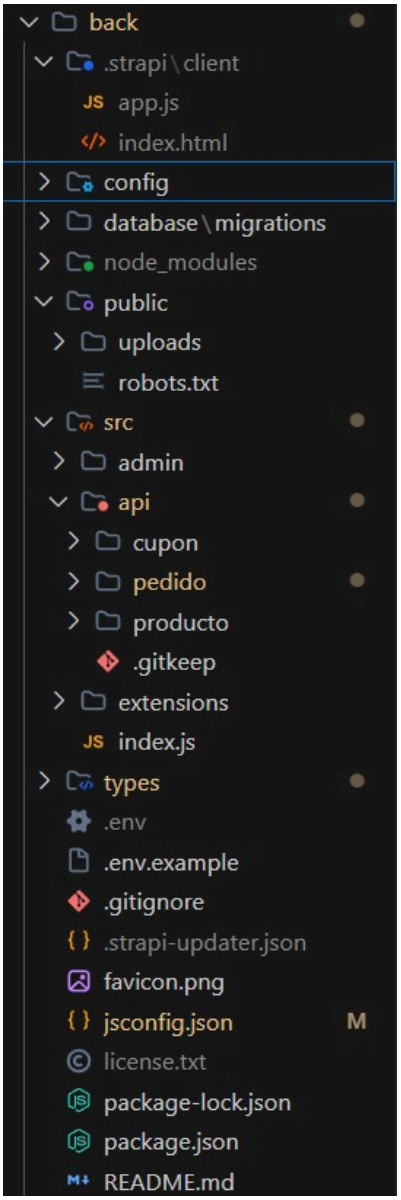



*Figura 6. Arquitectura del Backend.*

La arquitectura del backend se basa en el ecosistema de Strapi, organizando la lógica de negocio dentro de la carpeta src/api a través de módulos específicos para cada entidad, como productos y pedidos. Esta estructura utiliza controladores y servicios para gestionar las peticiones de forma independiente,

mientras que las carpetas de config y database aseguran la conexión con PostgreSQL y el manejo de variables de entorno.

**Back-auth**

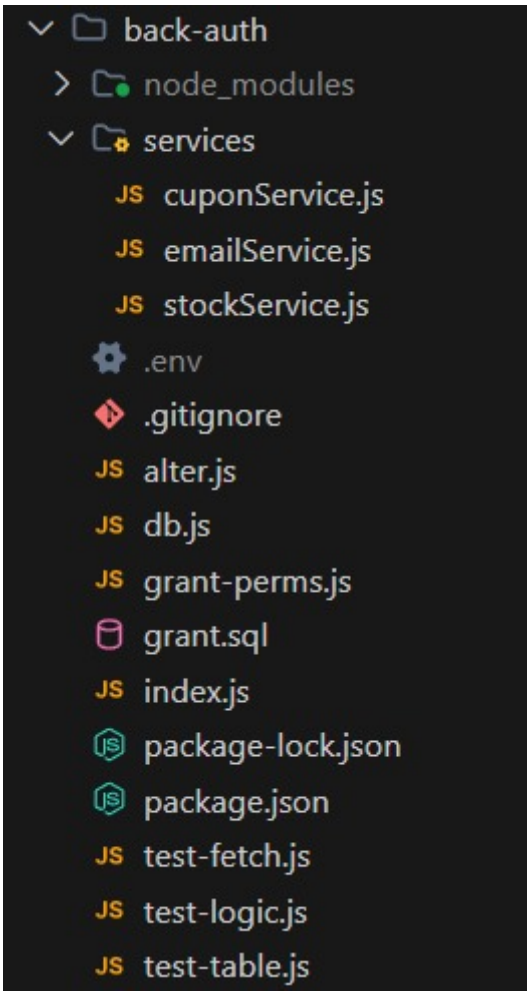



*Figura 7. Estructura del back-auth.*

La carpeta back-auth funciona como un microservicio especializado en Node.js, encargado de gestionar procesos críticos que exceden la administración de contenido base. Su estructura organiza la lógica de negocio en una carpeta

/services, centralizando allí la integración con Mercado Pago, el manejo de cupones y las notificaciones automáticas. Este módulo conecta la base de datos PostgreSQL con servicios externos para asegurar que el flujo de compra y la actualización de stock sean precisos y seguros.

## ARQUITECTURA DEL SISTEMA

**Informe detallado de las funciones de FRONTEND (/front)**

- **src/components/:**
  Esta carpeta aplica el principio de responsabilidad única. cada componente encapsula su propia lógica y estética.
  - Breadcrumbs.css/.jsx: Genera la ruta de navegación jerárquica procesando la URL actual, facilitando la ubicación del usuario.
  - FilterPanel.css/.jsx: Centraliza la lógica de búsqueda. Permite filtrar el catálogo por categorías y rangos de precio, comunicándose con la página de productos.
  - Footer.jsx/.html: El archivo .jsx actúa como contenedor reactivo que inyecta la estructura definida en el .html, centralizando información institucional y redes sociales.
  - Header.jsx/.html: Gestiona la navegación superior, el buscador y el acceso dinámico al carrito de compras.
  - ProductCard.css/.jsx: El componente más reutilizado. Recibe datos dinámicos del backend para renderizar la imagen, precio y descripción de cada mate.
  - ReviewForm.css/.jsx: Formulario interactivo que gestiona el estado de las reseñas antes de enviarlas a la base de datos.

- ReviewsModal.css/.jsx: Implementa una interfaz de “ventana emergente” para visualizar opiniones sin perder el contexto de la página actual.
- StarRating.css/.jsx: Lógica visual que traduce valores numéricos en iconos de estrellas para feedback rápido.
- Toast.css/.jsx: Sistema de notificaciones emergentes para avisos efímeros como “Producto agregado al carrito”.

- **src/pages/:**

  Orquestan la lógica de negocio y el consumo de las APIs de Strapi y el microservicio de autenticación.
  - Home.jsx/HomePage.jsx: Pantallas de aterrizaje con banners promocionales y acceso directo a productos destacados.
  - Productos.jsx: Vista principal del catálogo. Realiza la petición asíncrona a la API de Strapi y coordina el renderizado de las tarjetas de productos filtradas.
  - ProductPage.jsx: Ficha técnica detallada que muestra descripción extendida, stock real y la sección de reseñas.
  - Cart.jsx: Gestión visual del carrito. Calcula subtotales, totales y permite la edición de cantidades antes del checkout.
  - PagoDireccion.jsx/PagoEnvio.jsx/Pago-tarjeta.jsx: Un flujo fragmentado de compra que recolecta datos de logística y financieros de forma secuencial.
  - HistorialCompras.jsx: Consulta la base de datos PostgreSQL para listar pedidos anteriores vinculados al ID del usuario logueado.
  - UserProfile.jsx: Panel privado para la gestión de datos personales y estado de la sesión.
  - Favoritos.jsx: Filtra y muestra los productos que el usuario ha marcado para compra futura.

**Informe detallado de las funciones de BACKEND (/back)**

- **src/api/:**
  En esta carpeta se definen los modelos de datos y el comportamiento de los recursos principales del sistema.
    - cupon/: Gestiona la lógica de descuentos. Almacena códigos, porcentajes de rebaja y estados de validez para ser aplicados en el proceso de compra.
    - pedido/: Administra las transacciones del e-commerce. Se encarga de registrar las compras finalizadas, vinculando productos, usuarios y totales.
    - producto/: Define el catálogo de Mate Único. Contiene la información técnica de los mates y accesorios, precios, descripciones y la relación con las imágenes.
- **config/:**
  Determina el funcionamiento operativo del servidor y la comunicación con servicios externos.
    - admin.js: Configura el panel de administración de Strapi, permitiendo la gestión de usuarios administrativos.
    - api.js: Establece configuraciones generales para las respuestas de la API, como límites de carga y parámetros globales.
    - database.js: Archivo que define la conexión con PostgreSQL. Gestiona el pool de conexiones y los parámetros de acceso mediante variables de entorno.
    - middlewares.js: Controla las capas de seguridad, incluyendo las políticas de CORS y la seguridad de los activos multimedia.
    - plugins.js: Activa y configura extensiones adicionales.

  - server.js: Especifica el host, puerto y las llaves de seguridad necesarias para el arranque del servidor.
- **Archivos de Raíz y Entorno:**
  - .env.example: Plantilla de referencia para la configuración de variables de entorno necesarias para desplegar el sistema.
  - package.json: Manifiesto del proyecto que lista las dependencias del motor Strapi y los scripts de ejecución.

**Informe detallado de las funciones de BACK-AUTH (/back-auth)**

- **services/:** En esta carpeta se encuentra el “cerebro” operativo de las funcionalidades automáticas del e-commerce.
  - cuponService.js: Gestiona la validación compleja de cupones. Verifica en tiempo real si un código es válido, si expira o si ya superó el límite de usos permitidos.
  - emailService.js: Se encarga de la comunicación con el cliente. Automatiza el envío de correos electrónicos transaccionales, como la confirmación de recepción de pedido o avisos de cambio de estado.
  - stockService.js: Controla que no se realicen ventas sin existencias y actualiza las cantidades disponibles en la base de datos tras cada transacción confirmada.
- **Infraestructura y Datos:** Archivos encargados de la conexión y manipulación de la base de datos y permisos.
  - db.js: Configura el pool de conexiones a PostgreSQL. Permite que este microservicio realice consultas SQL de alto rendimiento de forma independiente a Strapi.

  - alter.js: Script de utilidad diseñado para realizar modificaciones estructurales rápidas o correcciones manuales en las tablas de la base de datos.
  - grant-perms.js: Automatiza la asignación de permisos dentro del ecosistema. Asegura que los roles de usuario tengan acceso exacto a las funciones que necesitan, reforzando la seguridad del sistema.
- **Archivos de Ejecución y Pruebas:** Puntos de entrada y herramientas de diagnóstico para asegurar la estabilidad del backend.
  - index.js: Es el punto de entrada principal del servidor Node.js. Levanta el servicio y pone en marcha los escuchadores para las peticiones del frontend.
  - test-fetch.js/test-logic.js/test-table.js: Archivos de pruebas unitarias y de integración. Sirven para verificar que la comunicación con la API, la lógica de negocio y la integridad de las tablas funcionen correctamente antes de pasar a producción.

## DEFINICIÓN DE ENDPOINTS DE API.

- **/api/cupon/:** Maneja los códigos de descuento y beneficios de la tienda.
  - GET /api/cupones: muestra la lista de todos los cupones existentes.
  - GET /api/cupones?filters[codigo][$eq]=:codigo: busca si el nombre de un cupón específico existe en la base de datos.
  - POST /api/auth/validate-cupon: (ubicado en back-auth/index.js). Envía el código al cuponService.js para verificar de forma segura si está vigente y tiene stock de usos disponible.
- **/api/pedido/:** Registra las ventas realizadas y permite al usuario consultar su actividad histórica.
  - POST /api/pedidos: Crea un nuevo registro de pedido en la base de datos una vez confirmado el pago.

- GET /api/pedidos?filters[user][id][$eq]=:userId: Filtra la colección de pedidos para devolver únicamente los que pertenecen al ID del usuario logueado. Es la base de la página de “Historial de Compras”.
- GET /api/pedidos/:id: Recupera la información detallada de una compra específica.
- POST /api/notify/order-confirmation: (ubicado en back-auth/index.js). Dispara el emailService.js para enviar el correo automático de confirmación al cliente.

- **/api/producto/:** Controla el catálogo de la tienda y la integridad del inventario disponible.
  - GET /api/productos: Obtiene todos los productos para mostrar en el catálogo. Soporta los filtros de precio y categoría del FilterPanel.
  - GET /api/productos/:id?populate=*: Obtiene la ficha técnica de un mate específico. El parámetro populate permite traer las imágenes y las reseñas asociadas.
  - POST /api/stock/verify: (ubicado en back-auth/index.js). Consulta al stockService.js para asegurar que el producto sigue disponible un segundo antes de procesar el pago.
  - PUT /api/productos/:id: Actualiza los datos del producto en Strapi.

## QA Y RESULTADOS DE PRUEBAS.

### Metodología y Configuración del Entorno

La validación del sistema Mate Único se rigió por la metodología de Pruebas de Caja Negra (Black Box Testing). Este enfoque permitió auditar la respuesta del sistema ante diversos estímulos del usuario, asegurándonos de que se cumpla con los requerimientos.

Para simular un escenario de producción real, se configuró un entorno de pruebas integral basado en los siguientes pilares:

- Conectividad y Seguridad: Se implementó la herramienta Ngrok para exponer los servicios locales a la red pública de forma segura. Esto fue esencial para habilitar protocolos HTTPS y permitir la comunicación con servicios externos, facilitando la recepción de Webhooks en tiempo real.
- Monitoreo de Arquitectura: Las pruebas validaron la interacción entre los diferentes módulos del sistema: la interfaz de usuario en React, el servicio de lógica y autenticación en Node.js y la gestión de contenidos en el CMS.
- Simulación de Transacciones: Se utilizó el entorno Sandbox de la pasarela de pagos. Esto permitió realizar ciclos completos de compra con datos de prueba, asegurando que el flujo de información, el procesamiento de descuentos y la actualización de inventario funcionen de manera consistente.

**Documentación de Evidencias**

Con el propósito de respaldar los resultados obtenidos, se ha llevado un registro detallado que incluye los procedimientos técnicos y el material de respaldo generado durante las pruebas.
Para revisar detalladamente las métricas y resultados de cada testeo se puede acceder al siguiente enlace: Planilla Casos de Prueba.

## INFORME DE DEFECTOS.

Durante el proceso de desarrollo y las instancias de revisión técnica, el sistema fue sometido a auditorías para identificar oportunidades de mejora y fallos de lógica. Este informe detalla las anomalías detectadas y los ajustes sugeridos por la cátedra, los cuales fueron solucionados para garantizar que el producto final cumpla con los estándares requeridos.

**Defectos Resueltos y Mejoras de Usabilidad.**

A continuación, describiremos las resoluciones más significativas aplicadas al sistema para optimizar la experiencia de usuario:

- **Gestión Inteligente de Inventario**: Se corrigió la falta de comunicación con el usuario ante la pérdida de stock mientras un producto permanece en el carrito. Ahora, el sistema bloquea el flujo de pago y notifica al cliente, evitando errores en la pasarela de pagos. También se restringió la visualización del stock total, mostrando únicamente alertas de “Últimas unidades” para mejorar la estética y la estrategia de venta.
- **Optimización de la Interfaz:** Se rediseñó la ubicación de la sección “Favoritos” para darle mayor visibilidad junto al carrito de compras. Además, se modificaron los botones de acción a la hora de realizar el checkout, utilizando términos más precisos como “Continuar al pago” para reducir la incertidumbre del cliente.
- **Lógica de Personalización y Privacidad:** Se implementó una lógica condicional para la opción de grabado láser, ocultándose automáticamente en productos cuyos materiales no son aptos para este proceso, como el vidrio. Por otro lado, se hicieron anónimas las reseñas de productos para proteger la identidad de los compradores, manteniendo la integridad del sistema de valoraciones.
- **Control de Estados de Logística:** Se eliminó la posibilidad de retroceder manualmente los estados de envío, asegurando un flujo lineal y coherente desde la preparación hasta la entrega final del pedido.

**Planilla Detallada de Defectos y Evolución.**

Para una revisión de cada uno de los puntos tratados, incluyendo la criticidad, prioridad y la secuencia de pasos para su validación, se ha realizado el siguiente registro:

Acceso a Documentación Técnica: Defectos.

## MANUAL DE USUARIO.

Este manual ha sido diseñado como una guía para orientar al usuario en la operación de la plataforma “Mate Único”. El objetivo principal es garantizar una experiencia de navegación fluida, intuitiva y segura, permitiendo que cualquier persona, independientemente de sus conocimientos técnicos previos, pueda interactuar con el sistema de manera eficiente.

**Instrucciones de Uso Paso a Paso.**

**Paso 1: Explorar el catálogo.**

Ingrese a la plataforma para visualizar la oferta completa de productos.

Deslice hacia abajo para descubrir las últimas novedades y productos destacados de la tienda.




*Figura 8. Pantalla principal (Home).*

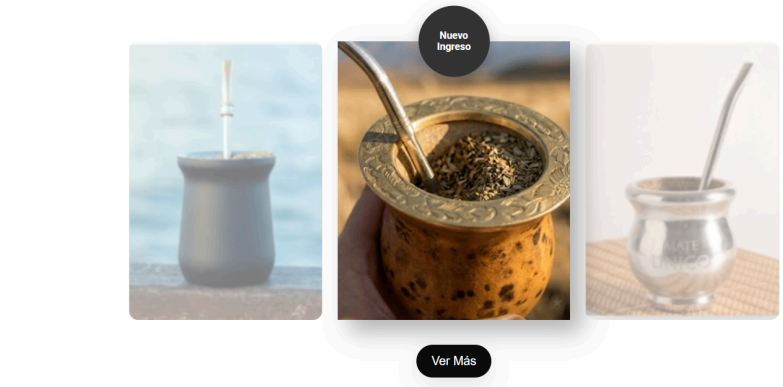



*Figura 9. Nuevos ingresos de la tienda.*

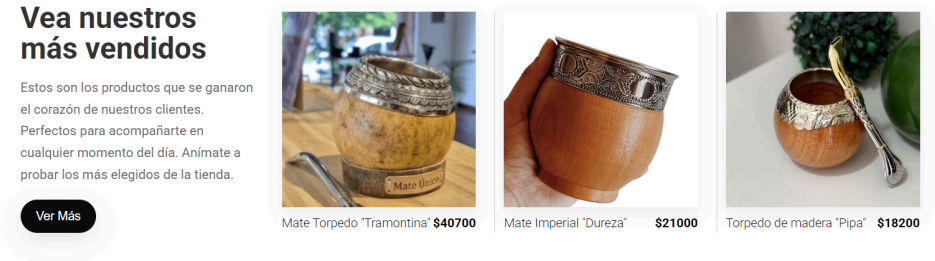



*Figura 10. Sección de productos más vendidos.*

**Paso 2: Acceder a su cuenta.**

Haga clic en el icono de usuario e ingrese con su correo electrónico. Esto le permitirá gestionar sus compras y guardar productos en favoritos.




*Figura 11. Formulario de inicio de sesión (Login).*

**Paso 3: Localizar productos específicos.**

Utilice la barra de búsqueda ubicada en la parte superior. Escriba el nombre del modelo que busca y el sistema le mostrará las coincidencias.

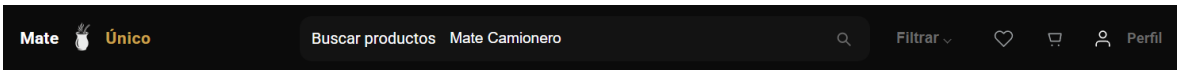



*Figura 12. Barra de búsqueda.*




*Figura 13. Coincidencias de la búsqueda.*

**Paso 4: Filtrar por categorías.**

Seleccione la opción de filtros ubicada en la parte superior y elija una categoría en el panel lateral para filtrar el catálogo  y visualizar únicamente el segmento de su interés.

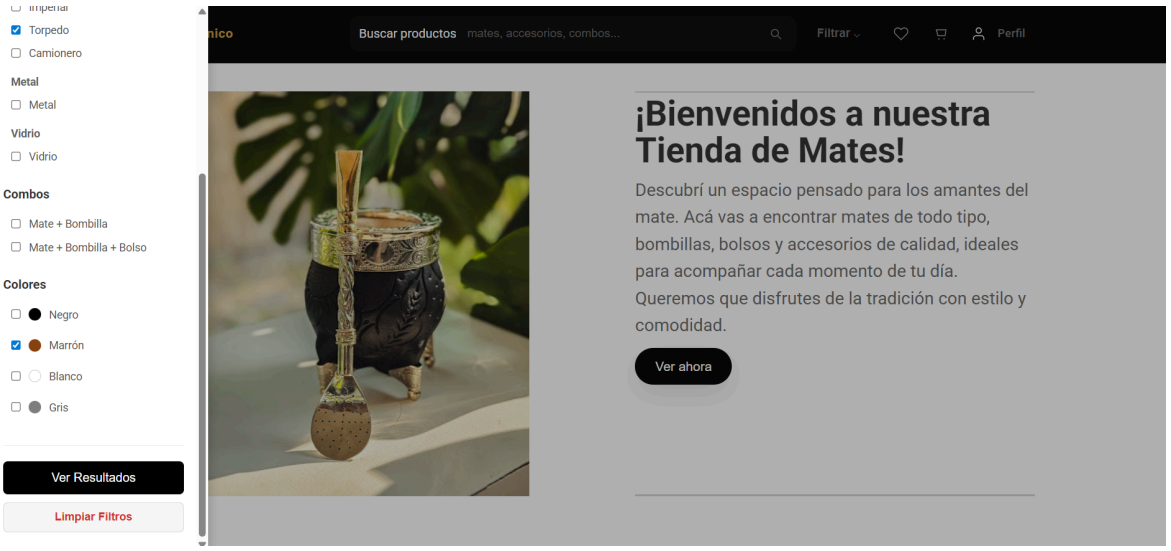



*Figura 14. Menú de filtros.*

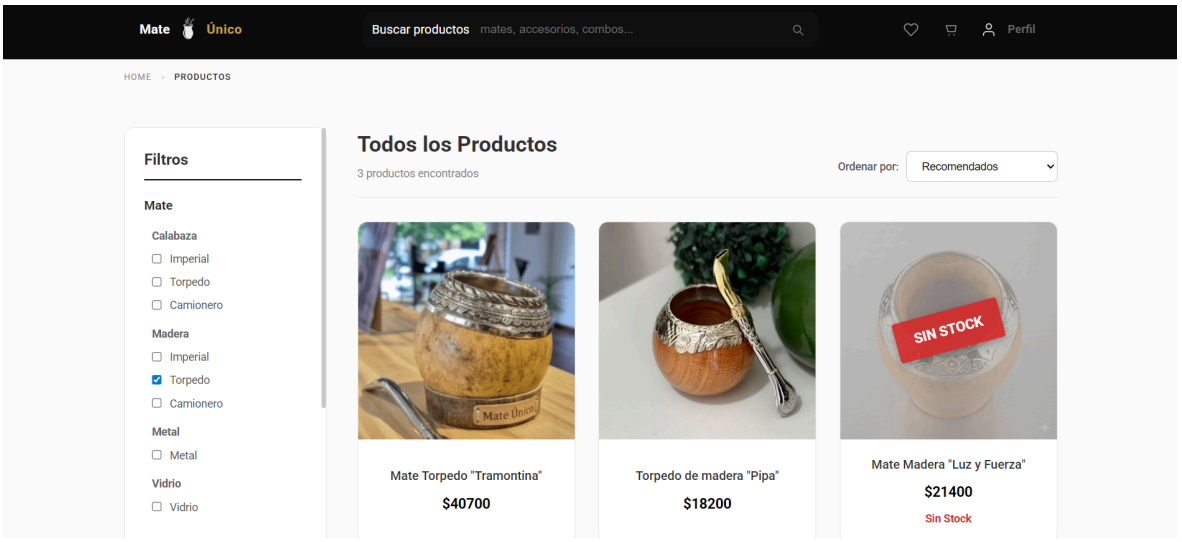



*Figura 15. Resultado de búsqueda por filtros.*

### Paso 5: Ordenar productos por precio

Utilice la función de ordenamiento para organizar el catálogo según su preferencia. Puede elegir visualizar los productos desde el precio más bajo al más alto, o viceversa, para facilitar su elección.

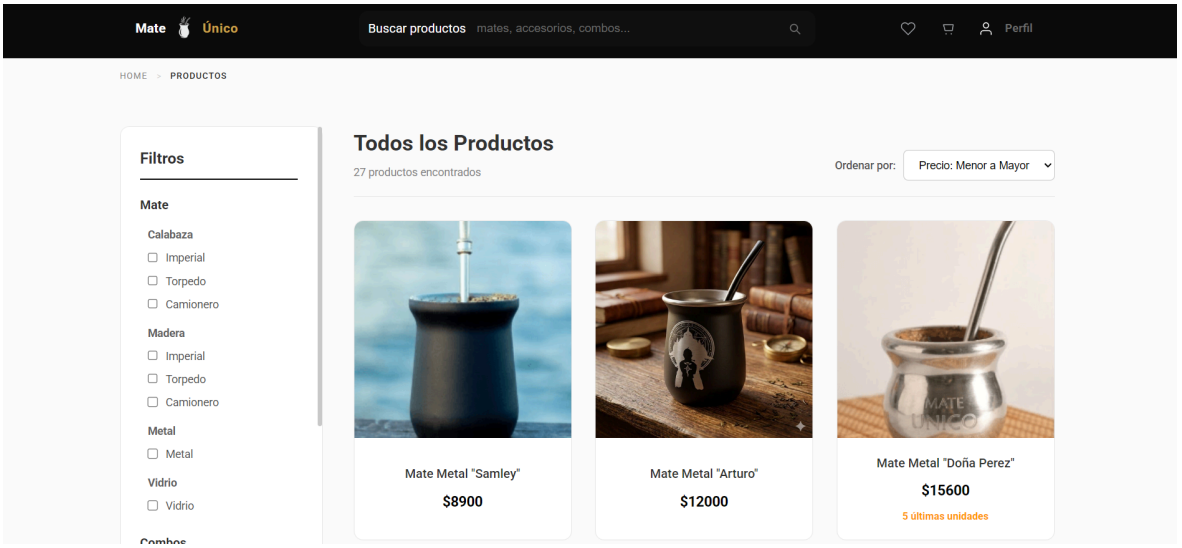



*Figura 16. Productos ordenados por precio de Mayor a Menor.*

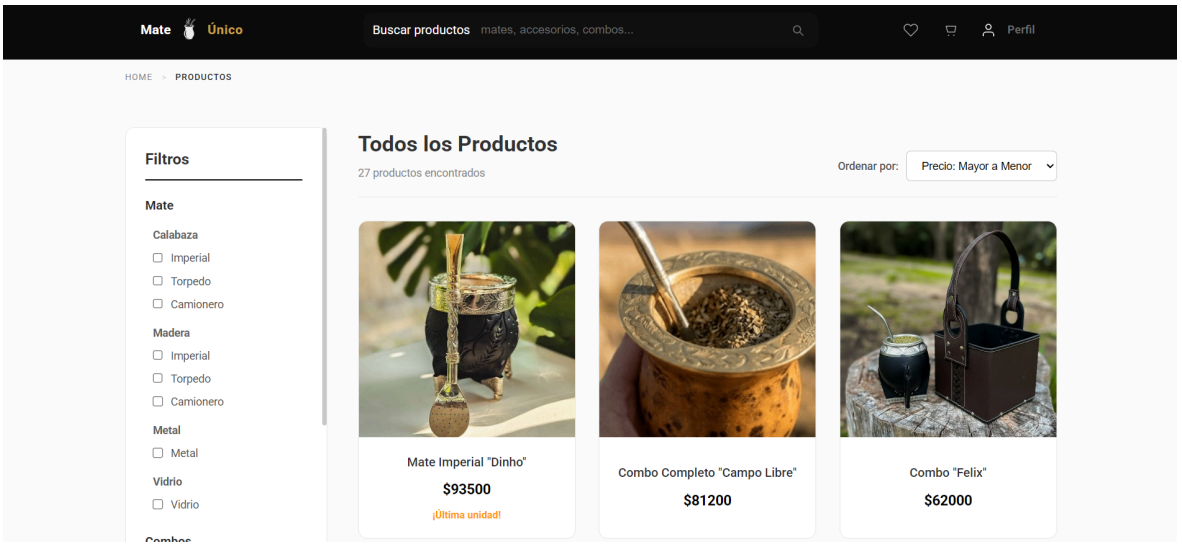



*Figura 17. Productos ordenados por precio de Menor a Mayor.*

**Paso 6: Consultar detalle de producto.**

Haga clic sobre la imagen del producto deseado para acceder a su ficha técnica. Aquí podrá leer la descripción detallada, materiales y ver más fotos.

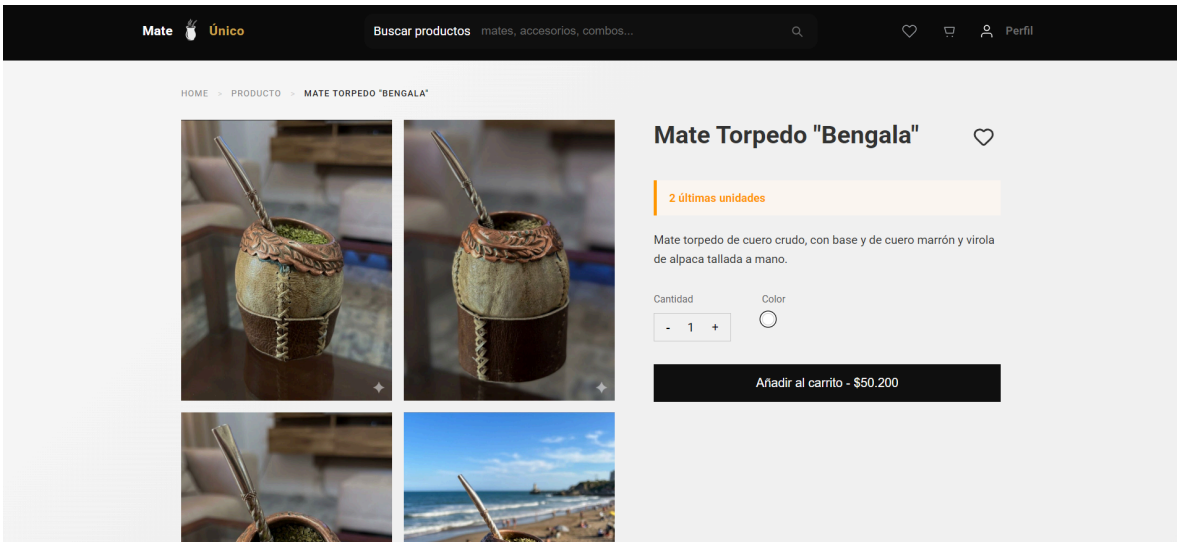



*Figura 18. Detalle de producto.*

**Paso 7: Personalizar su mate.**

Si el producto lo permite, complete la casilla de “Añadir Grabado”. Esta opción le permite añadir una personalización exclusiva a su mate antes de enviarlo al carrito.

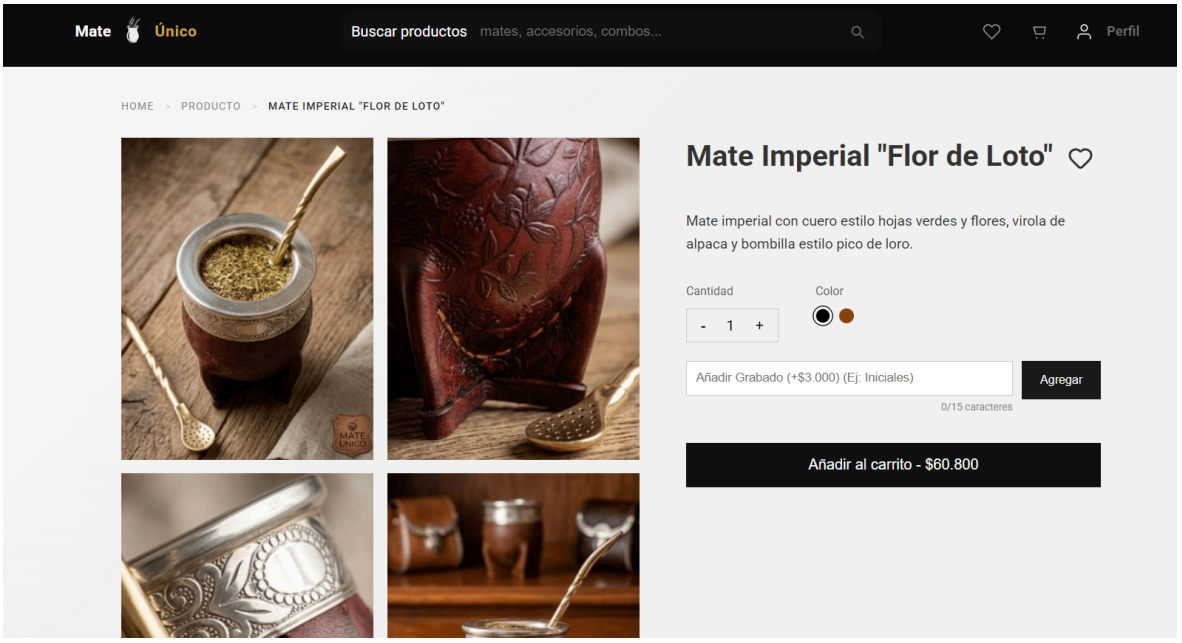



*Figura 19. Visualización de personalización de mate con grabado.*

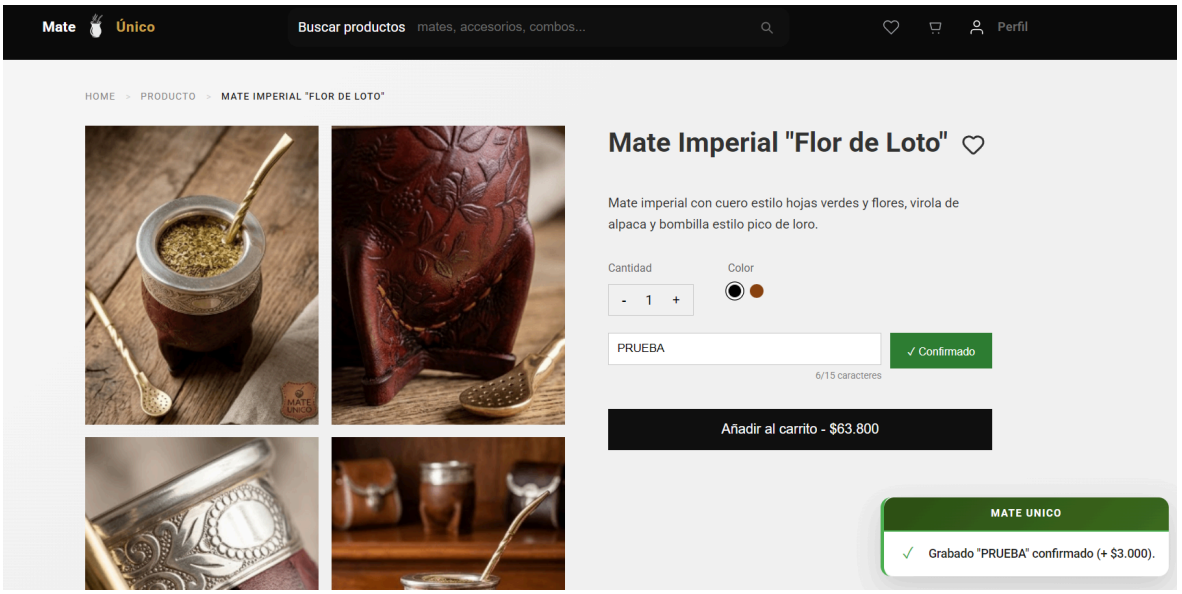



*Figura 20. Grabado en mate agregado exitosamente.*

**Paso 8: Guardar en Favoritos.**

Presione el icono del corazón en los productos que quiere añadir a su sección de favoritos. Acceda a su lista haciendo clic en el corazón ubicado en la parte superior, junto al carrito de compras.

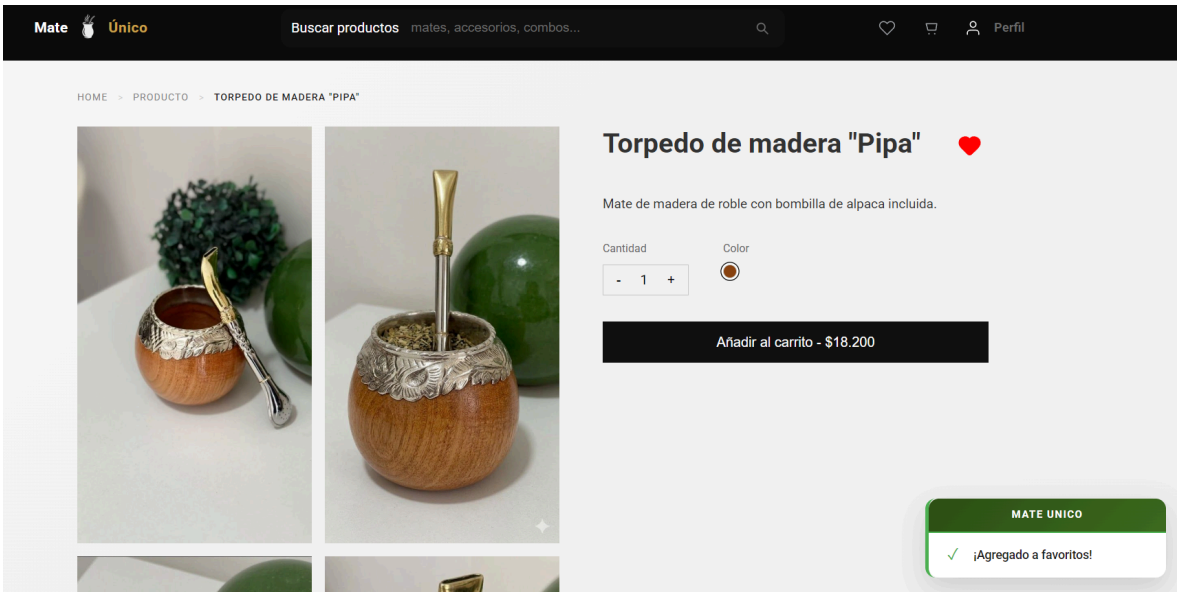



*Figura 21. Producto añadido a favoritos.*

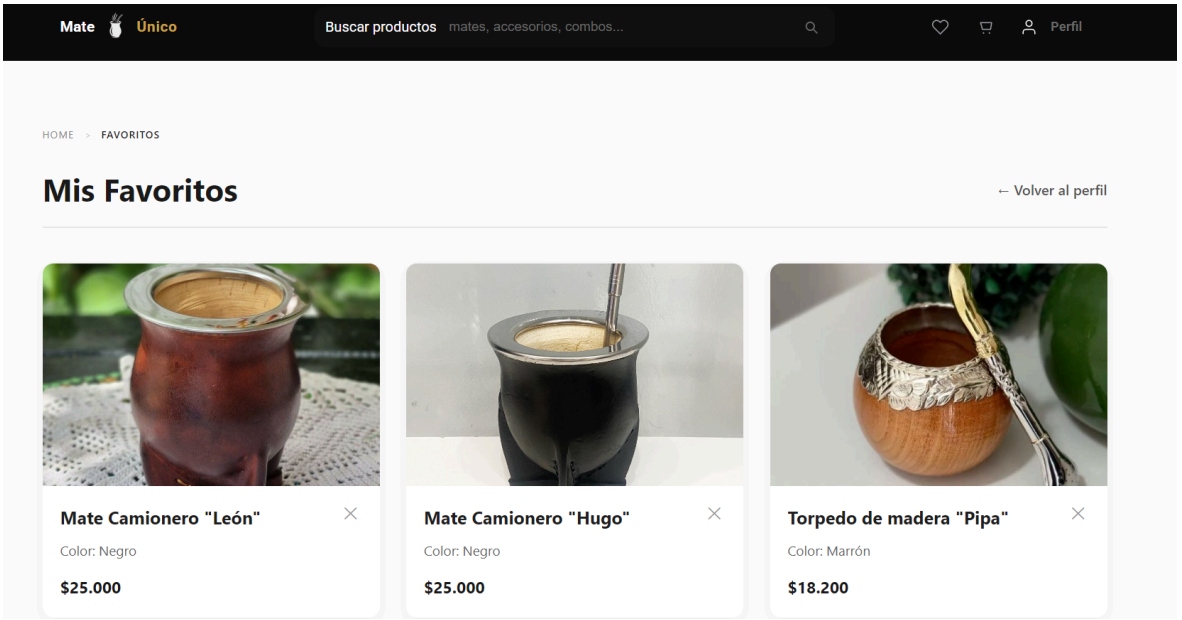


*Figura 22. Lista de favoritos.*

**Paso 9: Añadir productos al carrito de compras.**

Una vez seleccionado un producto, defina la cantidad de unidades que desea añadir al carrito utilizando el selector numérico. Luego, presione el botón “Añadir al carrito” para confirmar la selección y continuar con su compra.

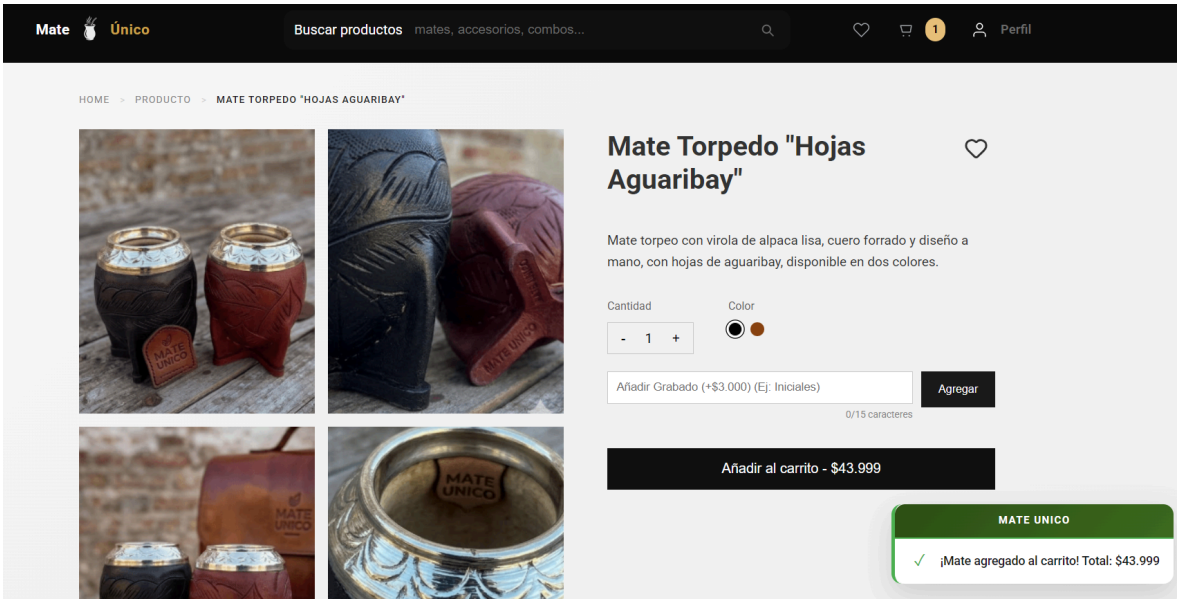



*Figura 23. Producto añadido al carrito.*

**Paso 10: Gestionar productos en el carrito de compras.**

Ingrese al carrito para revisar su selección. Desde aquí puede aumentar la cantidad de unidades de un mismo producto o eliminar aquellos que no desee adquirir.

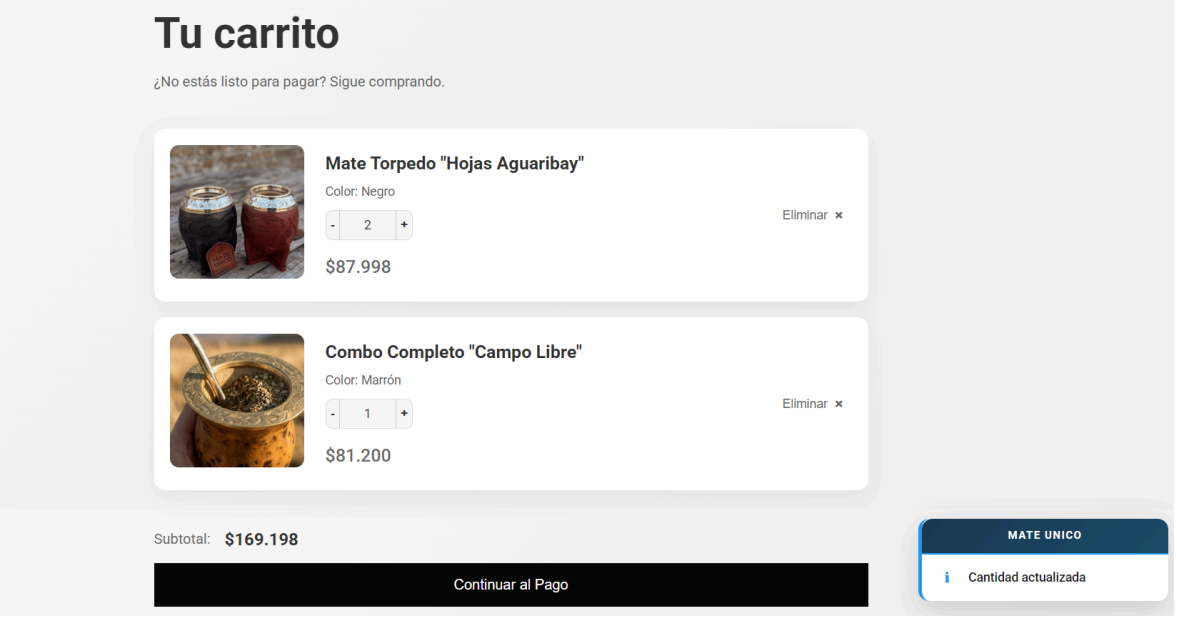



*Figura 24. Aumento de cantidades de un mate.*

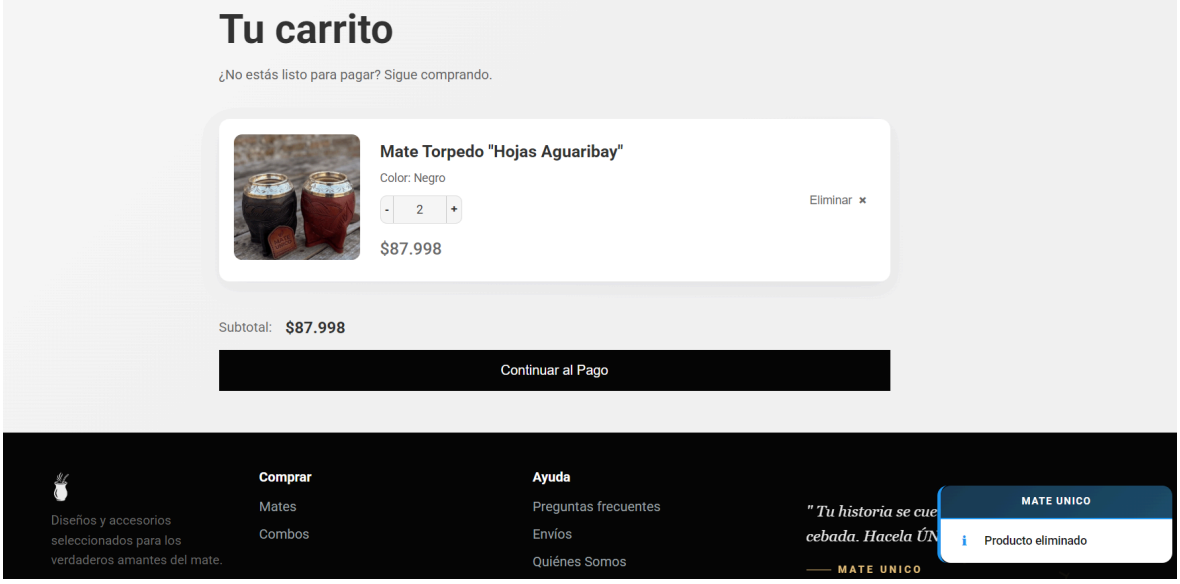



*Figura 25. Eliminación de un producto.*

**Paso 11: Iniciar el proceso de pago.**

Para proceder con la compra haga clic en el botón “Continuar al pago”. Esto lo llevará a la sección de Checkout donde podrá cargar sus datos y aplicar beneficios.

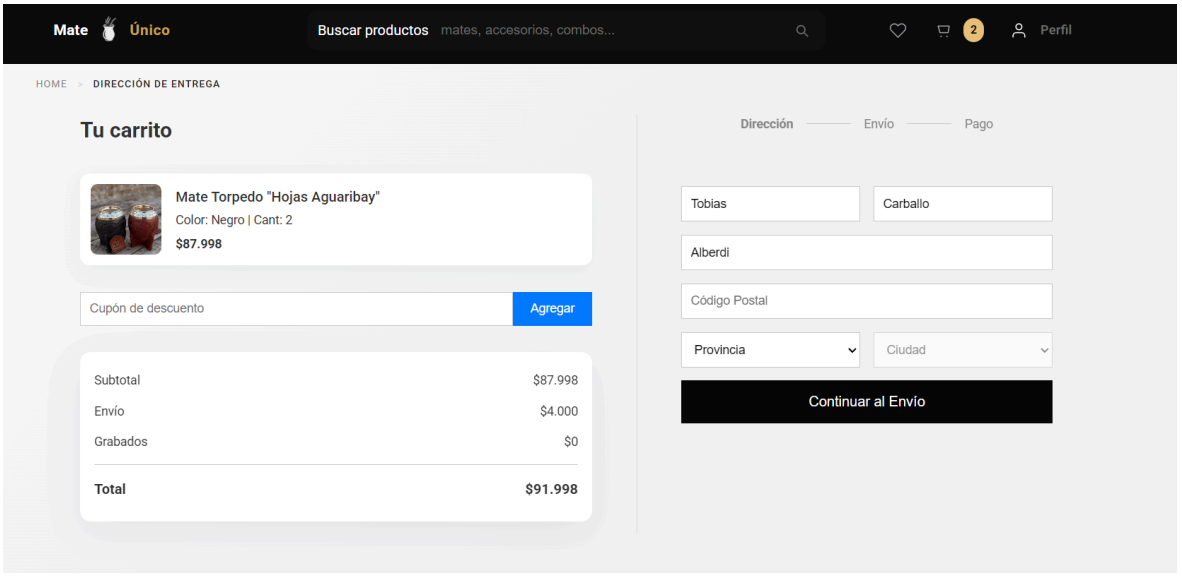



*Figura 26. Inicio del checkout y carga de datos.*

**Paso 12: Aplicar cupones de descuento.**

Escriba su código promocional en el campo “Cupón de descuento” y presione aplicar. Verifique que el descuento se haya reflejado correctamente en el total antes de continuar.

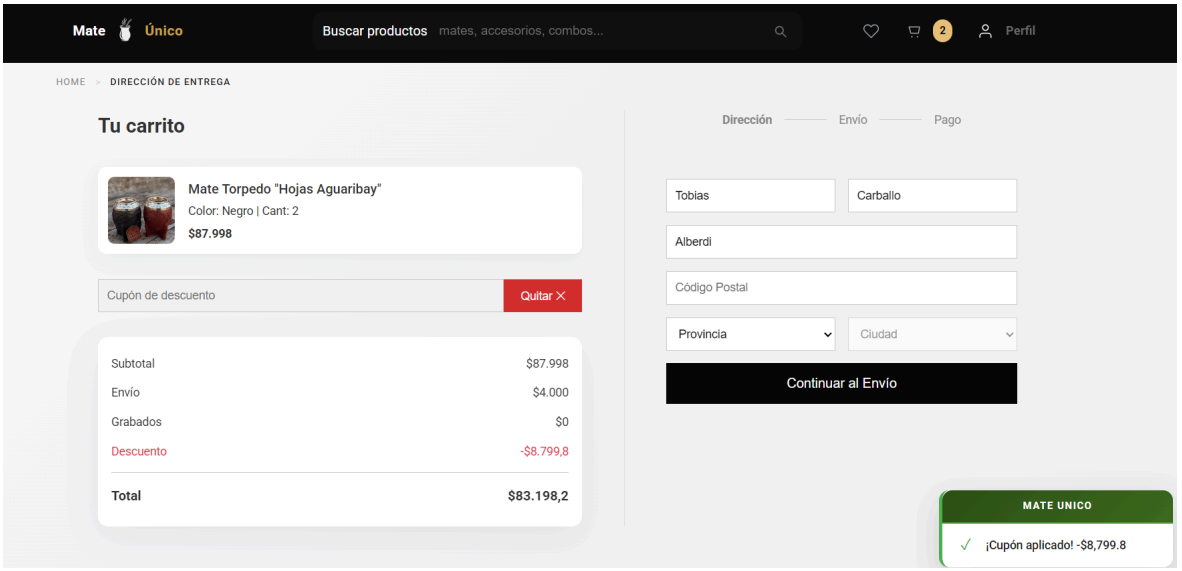



*Figura 27. Cupón aplicado en la compra.*

**Paso 13: Verificar alertas de disponibilidad.**

Preste atención a los avisos de stock. Si un producto se agota mientras está en su carrito, deberá removerlo para que el sistema le permita avanzar hacia el proceso de pago.

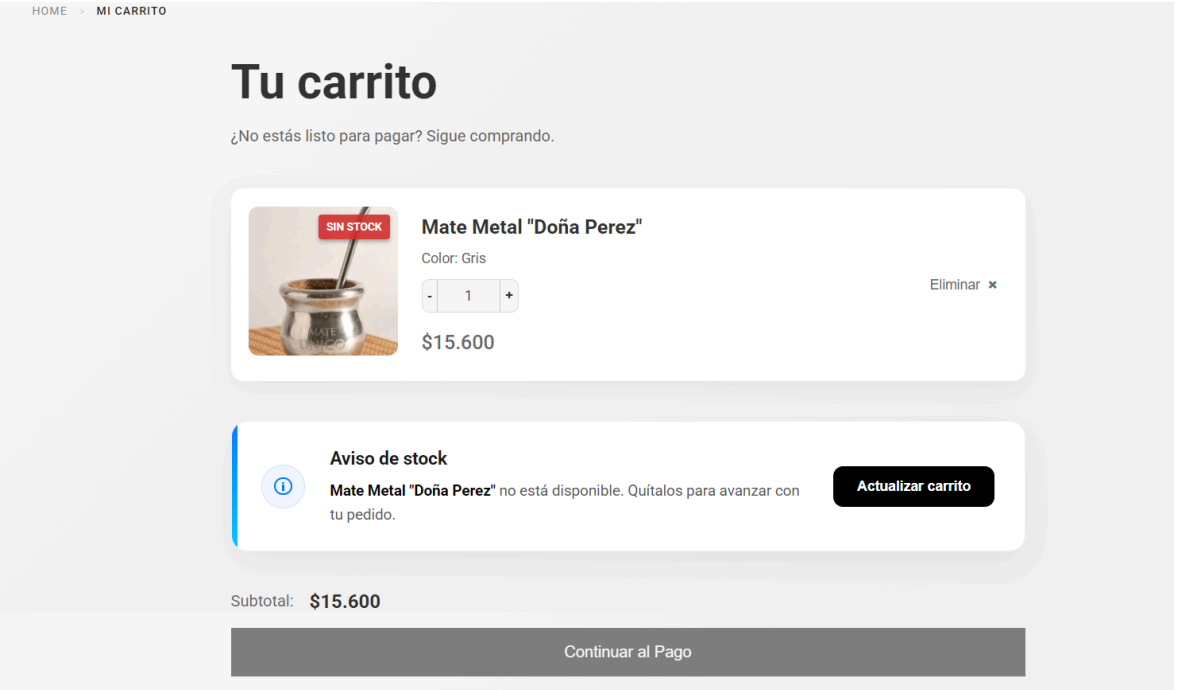



*Figura 28. Mensaje de advertencia por falta de stock.*

**Paso 14: Completar datos de envío.**

Complete el formulario con su nombre y datos de domicilio. Asegúrese de que la información sea precisa para garantizar una entrega exitosa.

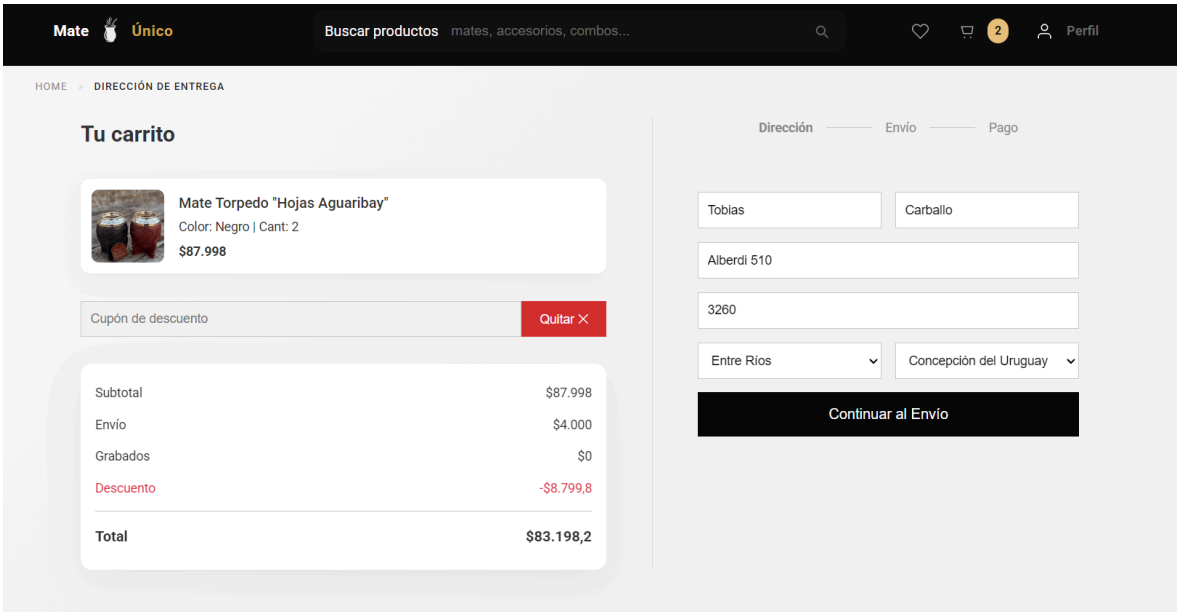



*Figura 29. Formulario de datos de envío completo.*

**Paso 15: Selección de tipo de envío.**

Una vez completado los datos, presione “Continuar al Envío” para seleccionar el método de envío que prefiera. El sistema calculará el costo de forma automática y lo sumará al total de su orden.

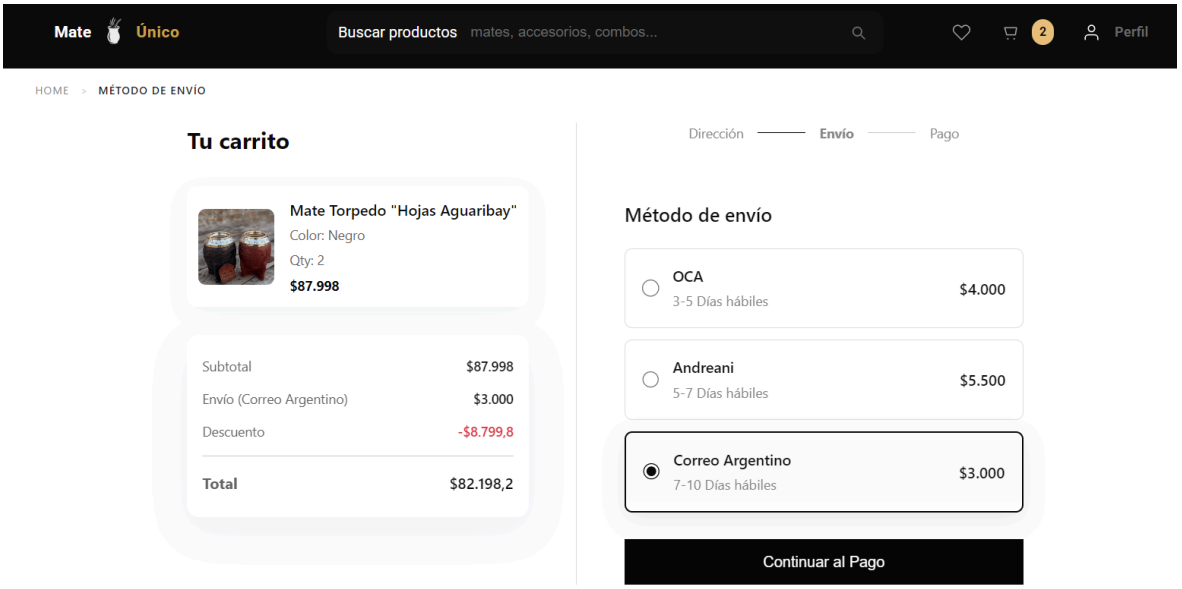



*Figura 30. Opciones de envíos y sus costos.*

**Paso 16: Proceder el pago seguro.**

Haga clic en “Continuar al Pago”. Será redirigido a una plataforma segura donde podrá elegir la forma de pago.

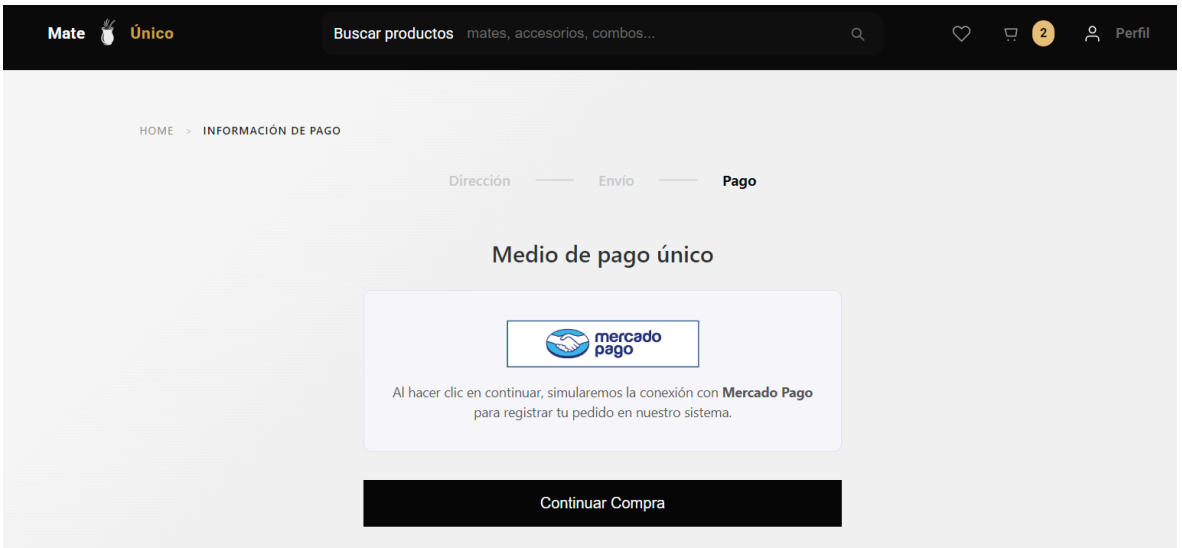


*Figura 31. Pantalla de selección de pago (Mercado Pago).*

**Paso 17: Compra confirmada.**

Una vez finalizado el pago, aguarde a la pantalla de confirmación. Recibirá un mail con los detalles de su pedido.

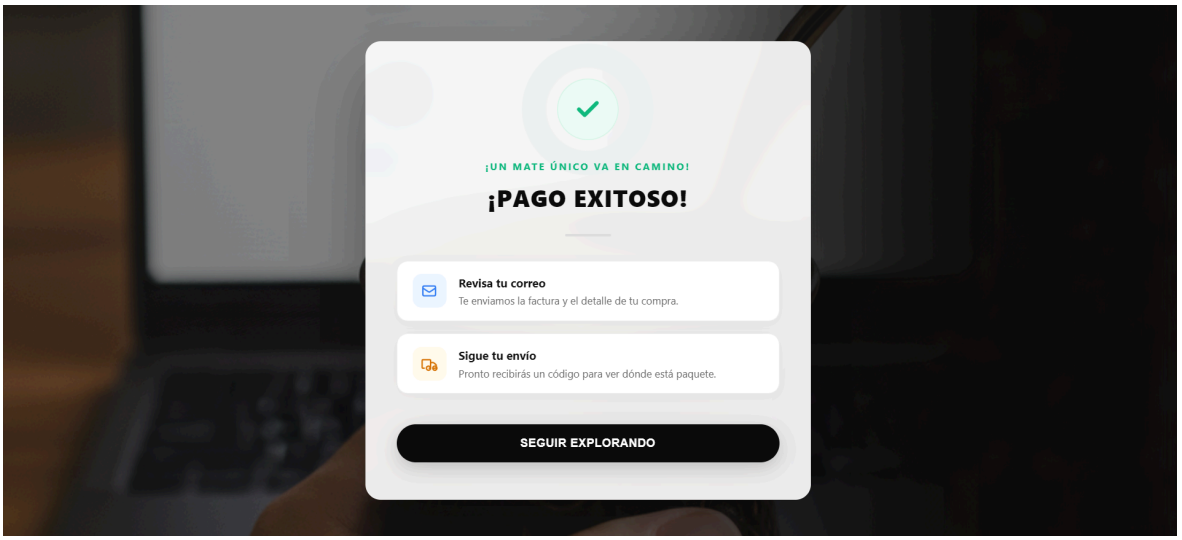



*Figura 32. Pantalla de confirmación de compra.*

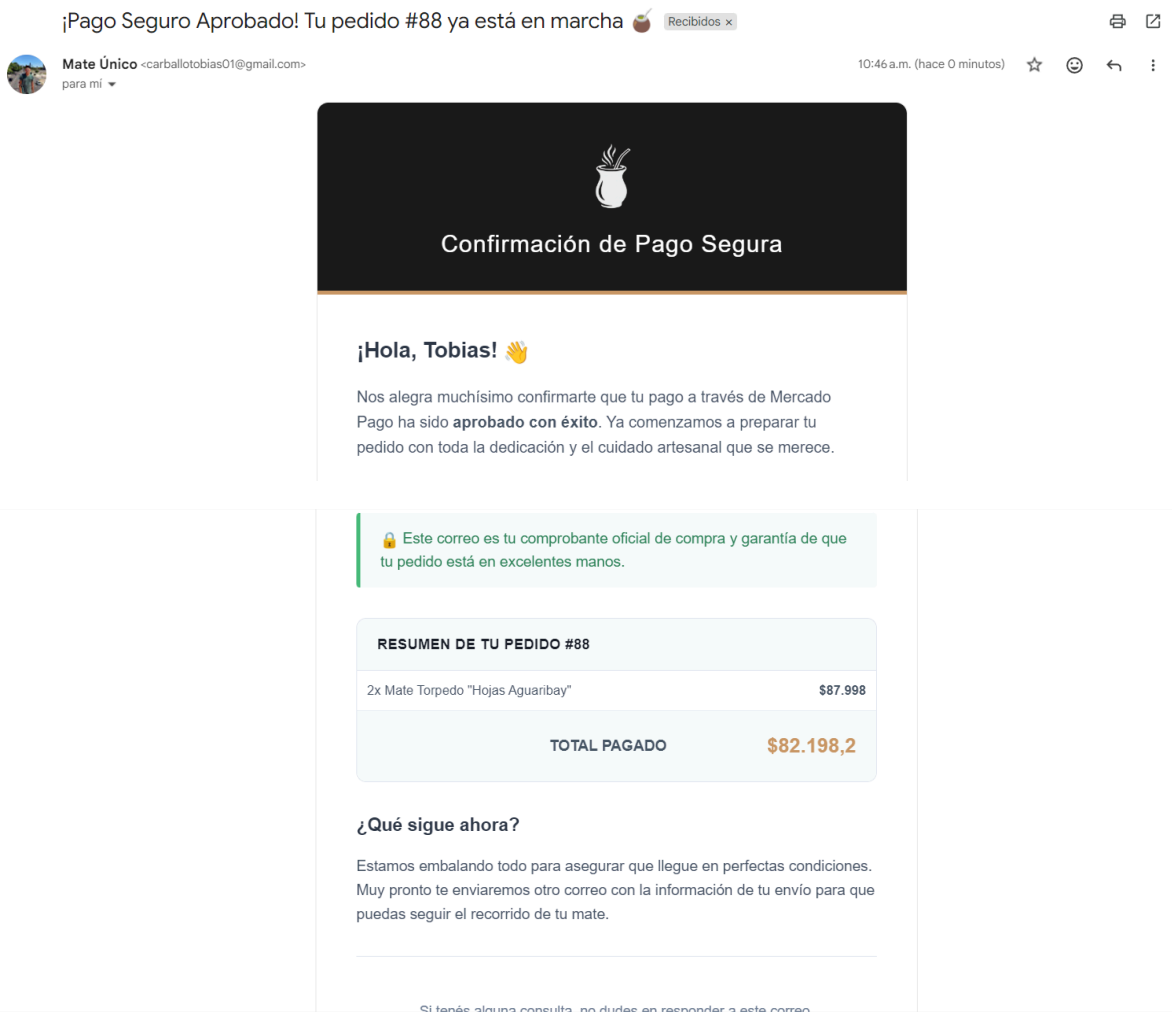



*Figura 33. Mail con detalles de la compra.*

**Paso 18: Consultar historial de pedidos.**

Diríjase a su perfil y seleccione “Historial de compras”. Allí podrá revisar el estado de sus pedidos actuales y consultar el detalle de todas sus transacciones pasadas.

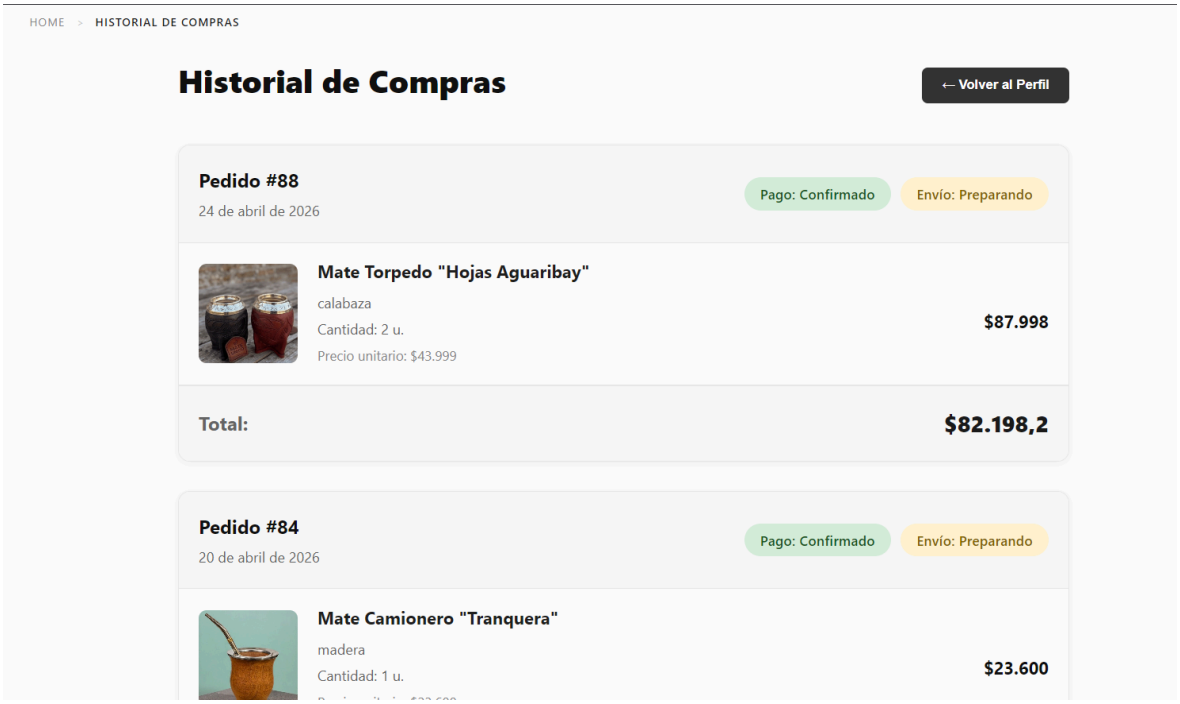



*Figura 34. Historial de Compras.*

**Paso 19: Valorar el producto.**

Una vez recibido su pedido, califique el producto asignando estrellas y un comentario. Su reseña se publicará de forma anónima para ayudar a otros usuarios.

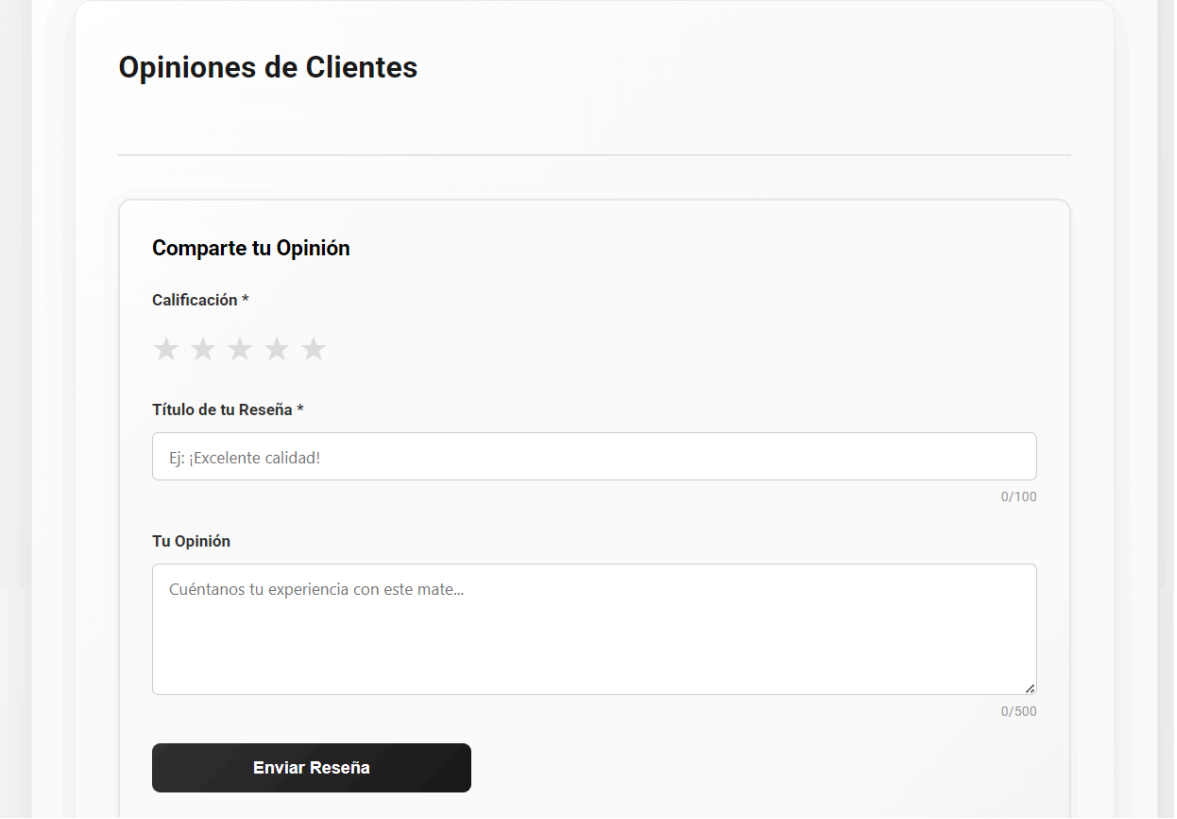



*Figura 35. Sección de reseñas.*

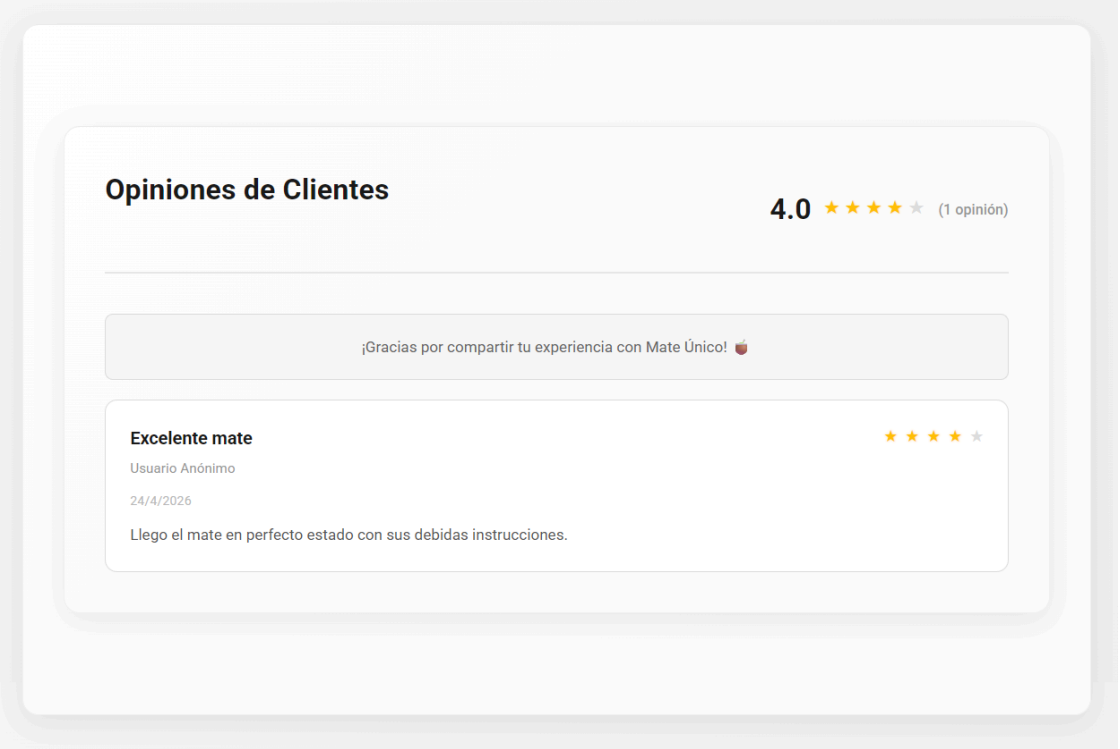



*Figura 36. Opinión y calificación.*

**Paso 20: Soporte y contacto.**

Si necesita asistencia, diríjase al pie de página del sitio. Allí encontrará los canales de contacto y redes sociales para recibir atención personalizada.




*Figura 37. Pie de página (Footer).*

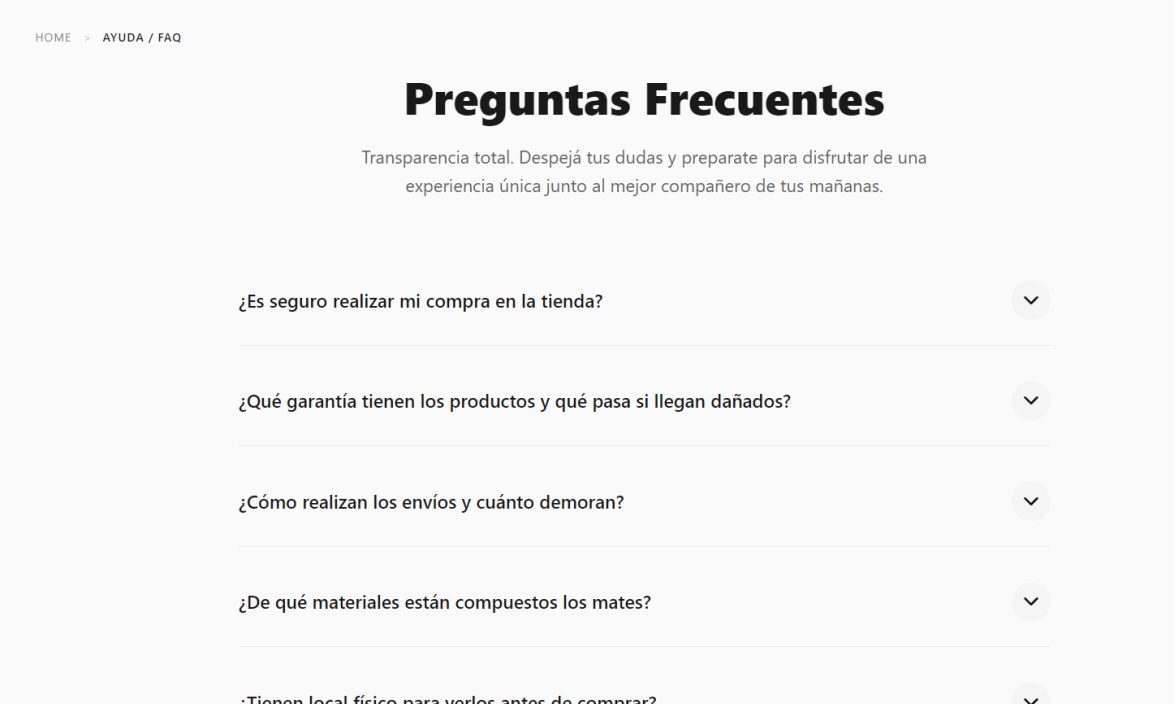



*Figura 37. Sección de Preguntas Frecuentes.*

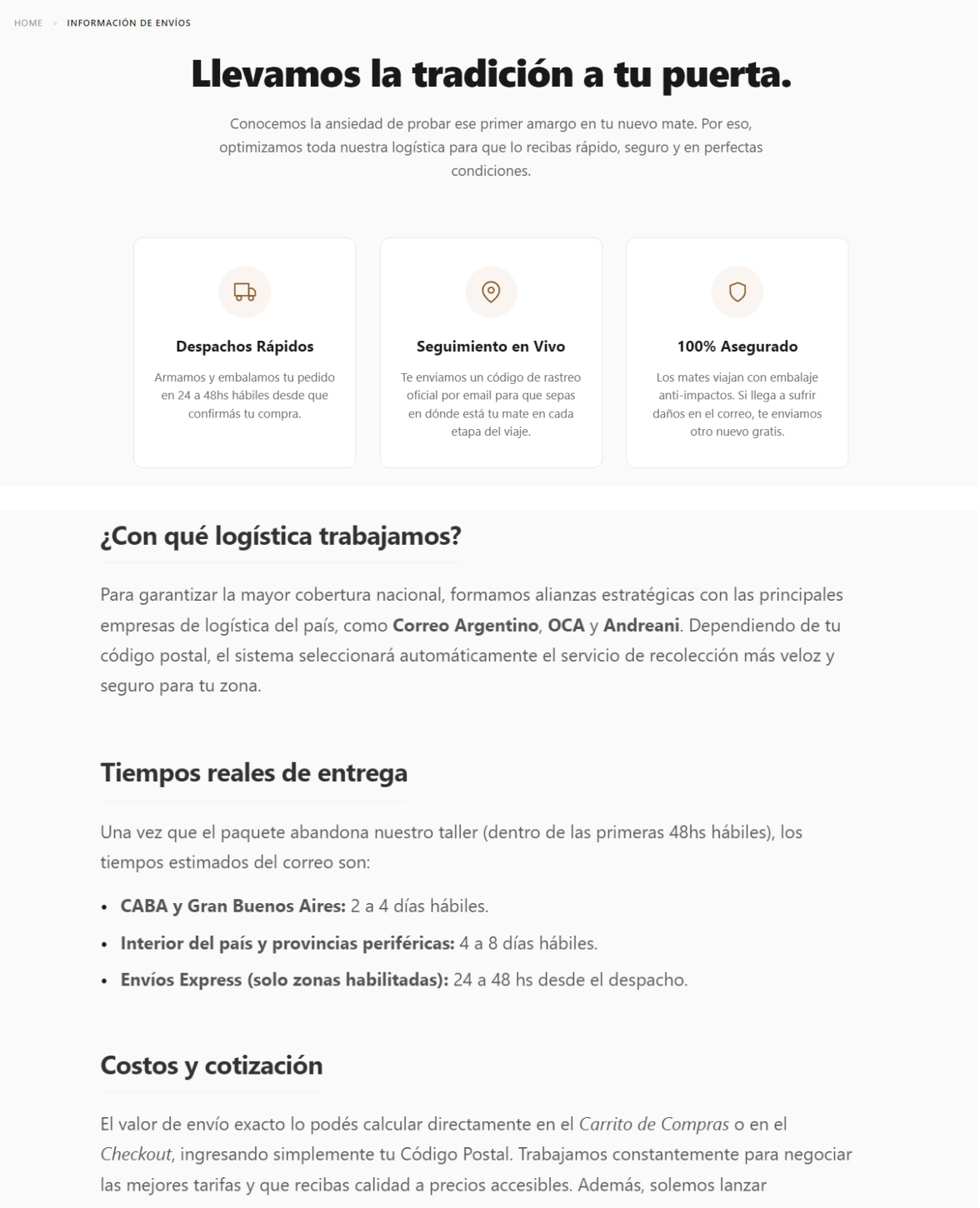



*Figura 37. Sección de Envíos.*

HOME > QUIÉNES SOMOS

# No hacemos solo mates. Compartimos historias.

*Creemos que detrás de cada infusión hay una conversación, una pausa y un momento para conectar. Esa es nuestra verdadera materia prima.*



NUESTRA HISTORIA

## Santino, Tobías y Lucrecia

Mate Único no nació en una junta de negocios ni en una oficina corporativa; nació, irónicamente, ***tomando mates***. Los tres compartíamos la misma frustración: encontrar un mate que no solo mantenga bien el calor y tenga buenos materiales, sino que además sea una pieza de diseño digna de llevar a todos lados.

Lo que empezó como una pequeña búsqueda personal de calabazas de paredes gruesas y fundas de cuero legítimo, se convirtió en una obsesión por el estándar de calidad más riguroso del mercado.

Hoy, seleccionamos, pulimos, forramos y empaquetamos cada mate con la misma pasión que le poníamos al primero. Cuidamos cada detalle porque sabemos perfectamente que un mate nunca es "sólo" un mate; va a ser, literalmente, el mejor compañero de tus mañanas por muchísimos años.

🌿 **Materiales Nobles**
Elegimos a mano cada pieza

🤲 **Artesanía Real**
Terminaciones cuidadas y

🤝 **Comunidad**
Conectamos desde la empatía.

*Figura 37. Sección de Quiénes Somos.*

## MANUAL DEL ADMINISTRADOR (GESTIÓN CMS)

Esta sección detalla los procedimientos técnicos para la administración de la plataforma a través del panel del control de Strapi. El administrador podrá gestionar el catálogo de productos, administrar cupones de descuento y supervisar el estado de los pedidos realizados.

**Paso 1: Acceso al panel administrativo.**

Ingrese sus credenciales de administrador en el panel de acceso.

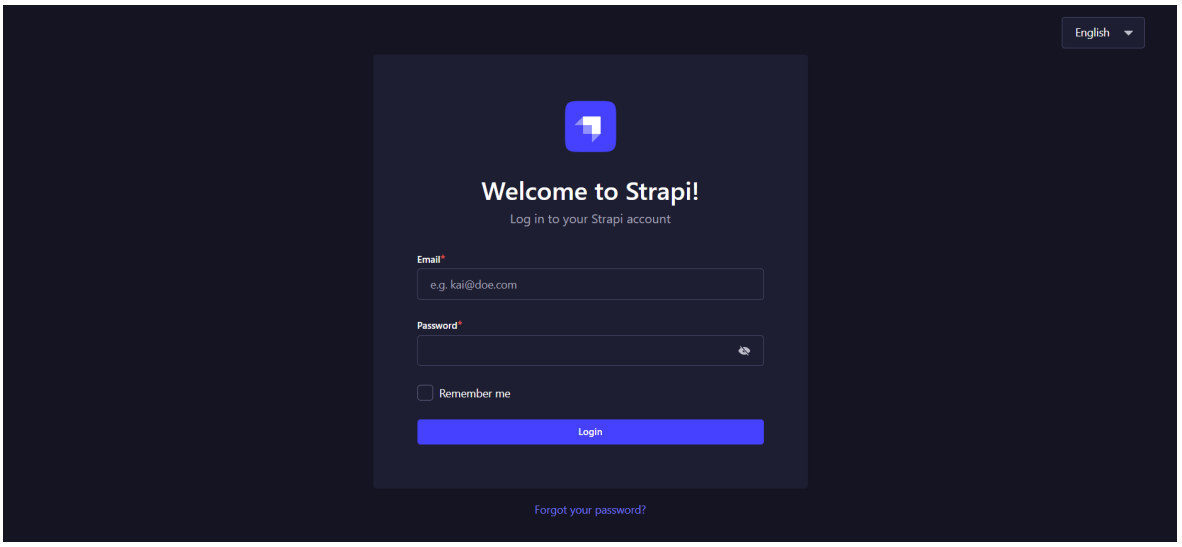



*Figura 38. Pantalla de login de Strapi.*

**Paso 2: Gestión de productos.**

Dentro de la colección “Productos”, podrá visualizar la lista completa de artículos. Utilice el botón “Create new entry” para añadir nuevos modelos o seleccione uno existente para editar su información.




*Figura 39. Listado de productos en el Content Manager.*

**Paso 3: Control de inventario y “Últimas unidades”.**

Actualice la cantidad de stock disponible. Cuando el número sea bajo, el sistema activará automáticamente en la web el aviso de “Últimas unidades”, incentivando la compra y alertando al usuario.

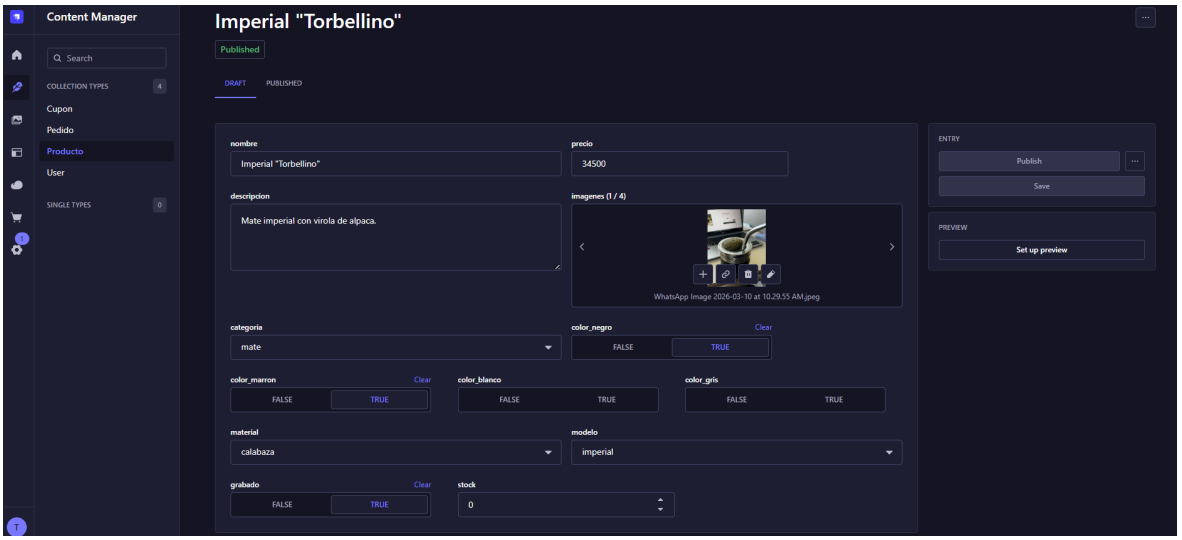



*Figura 40. Campo de edición de “stock” en un producto específico.*

**Paso 4: Configuración de materiales y grabado.**

Seleccione el material de cada producto y si admite grabado o no.

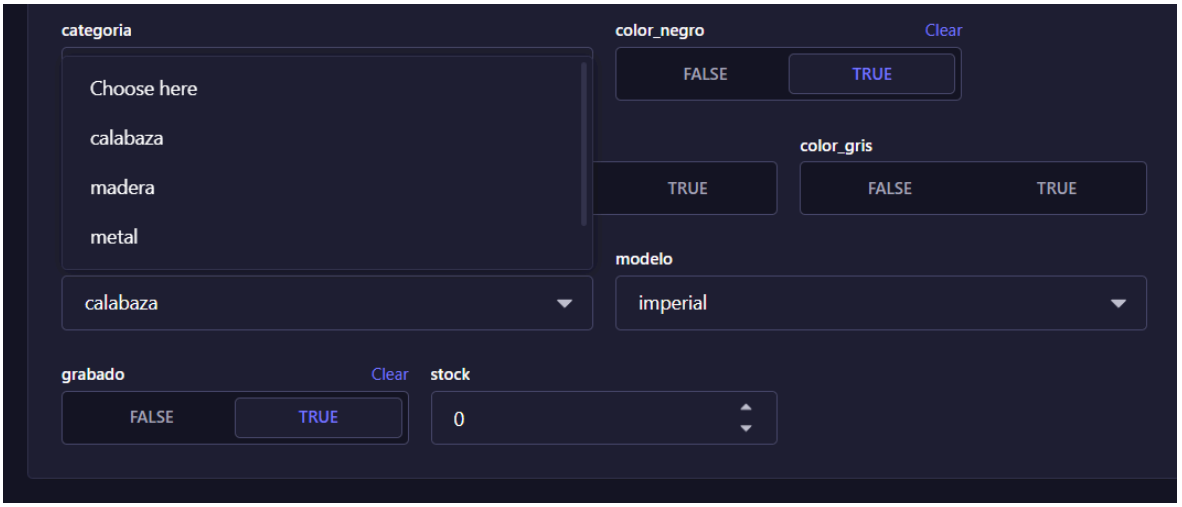



*Figura 41. Selector de material y grabado.*

**Paso 5: Administrador de cupones de descuento.**

Desde la sección de “cupones” puede crear códigos promocionales. Defina el porcentaje de descuento y la fecha de expiración, una vez cumplido el plazo, el cupón dejará de funcionar automáticamente.

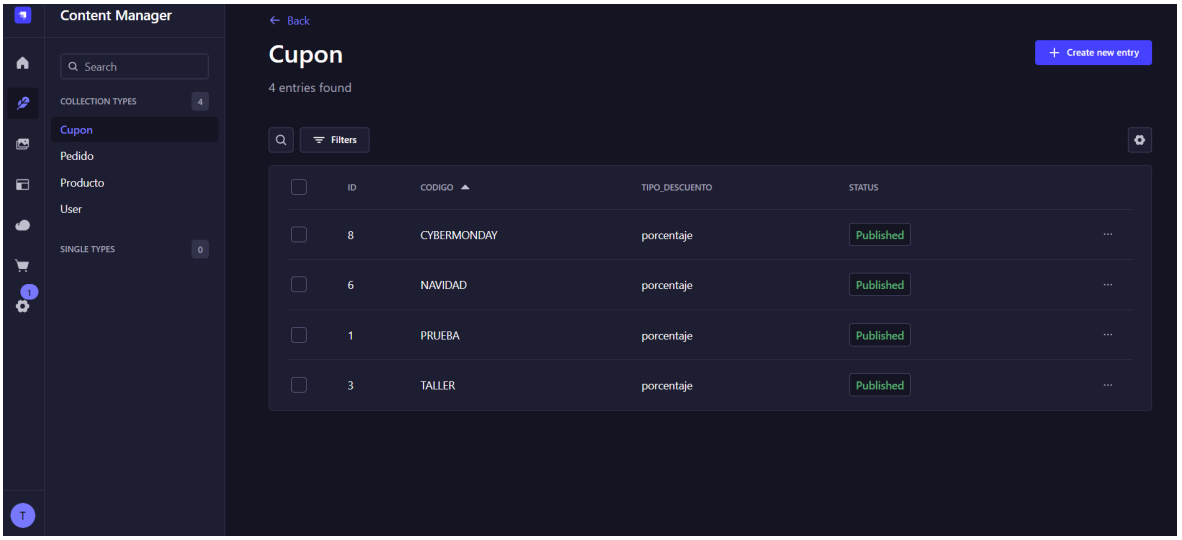



*Figura 42. Lista de cupones cargados.*

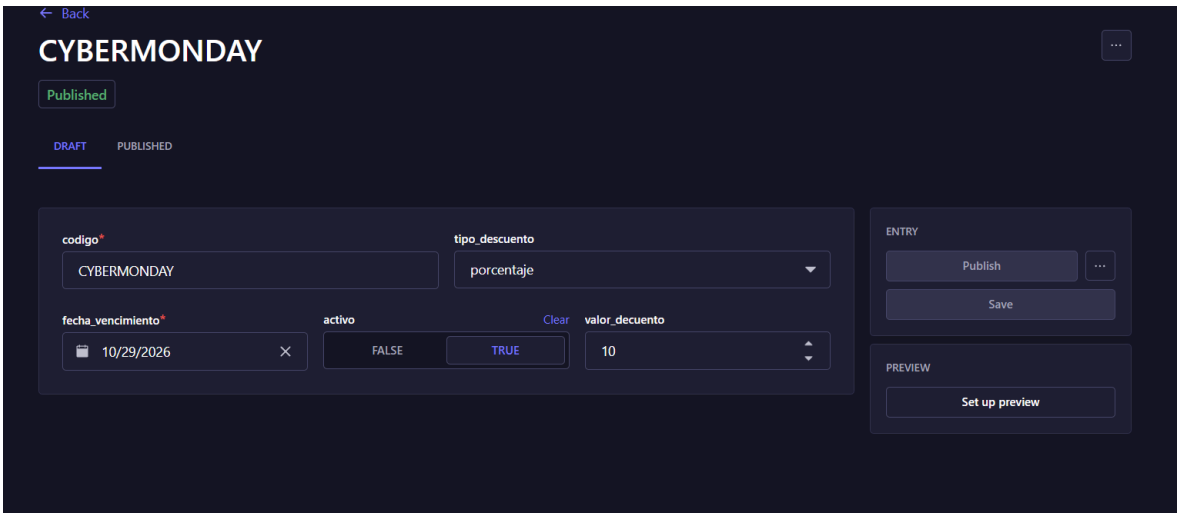



*Figura 43. Campo para crear cupones.*

**Paso 6: Monitoreo de Pedidos.**

Acceda a la sección de pedidos para ver las ventas realizadas. Aquí podrá consultar los datos del cliente, los productos adquiridos, el total abonado y el método de envío seleccionado.

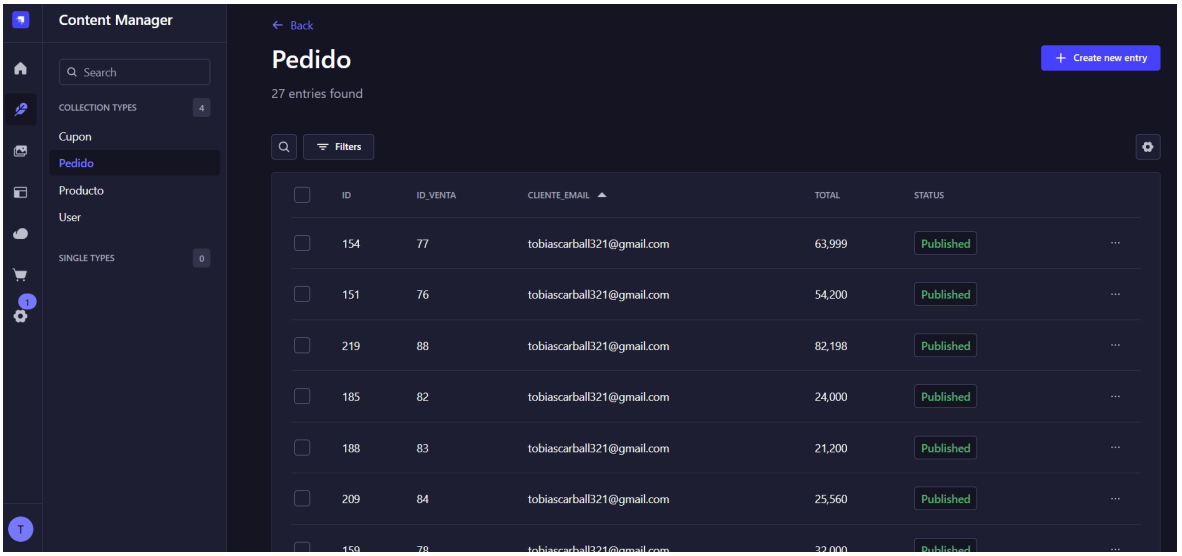



*Figura 44. Listado de pedidos.*

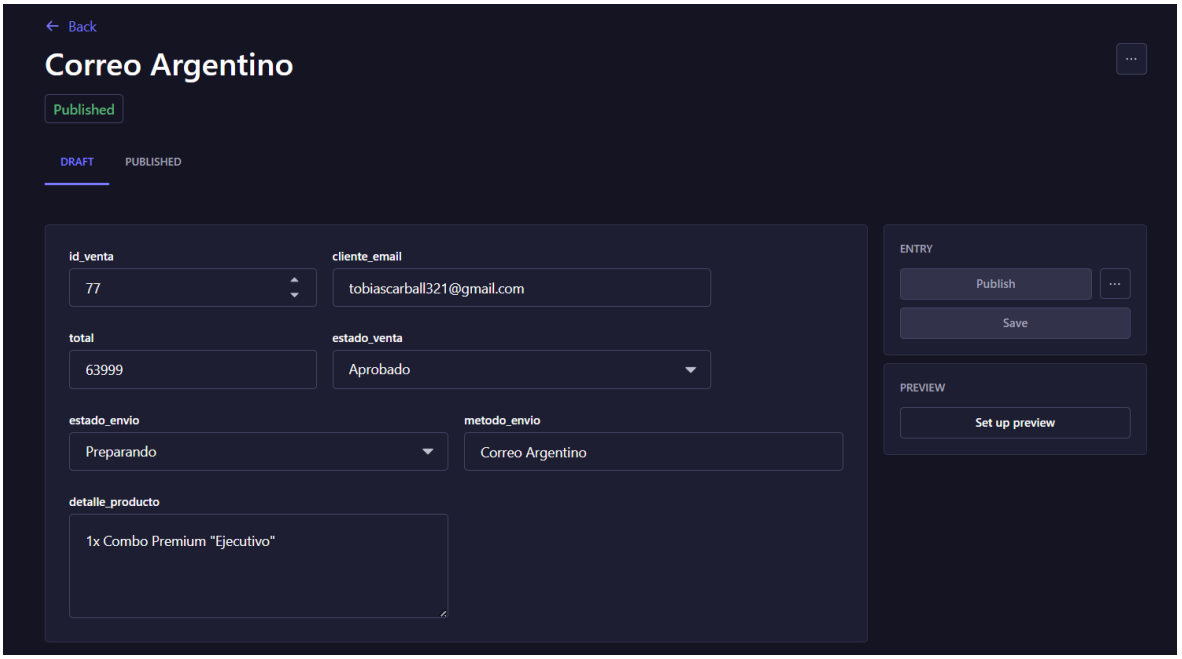



*Figura 45. Detalle de pedido.*

**Paso 7: Gestión de estados de envíos.**

Actualice el estado del pedido conforme avance la logística. El flujo es lineal: una vez marcado como entregado, el sistema bloquea cambios posteriores para garantizar la integridad de la transacción.

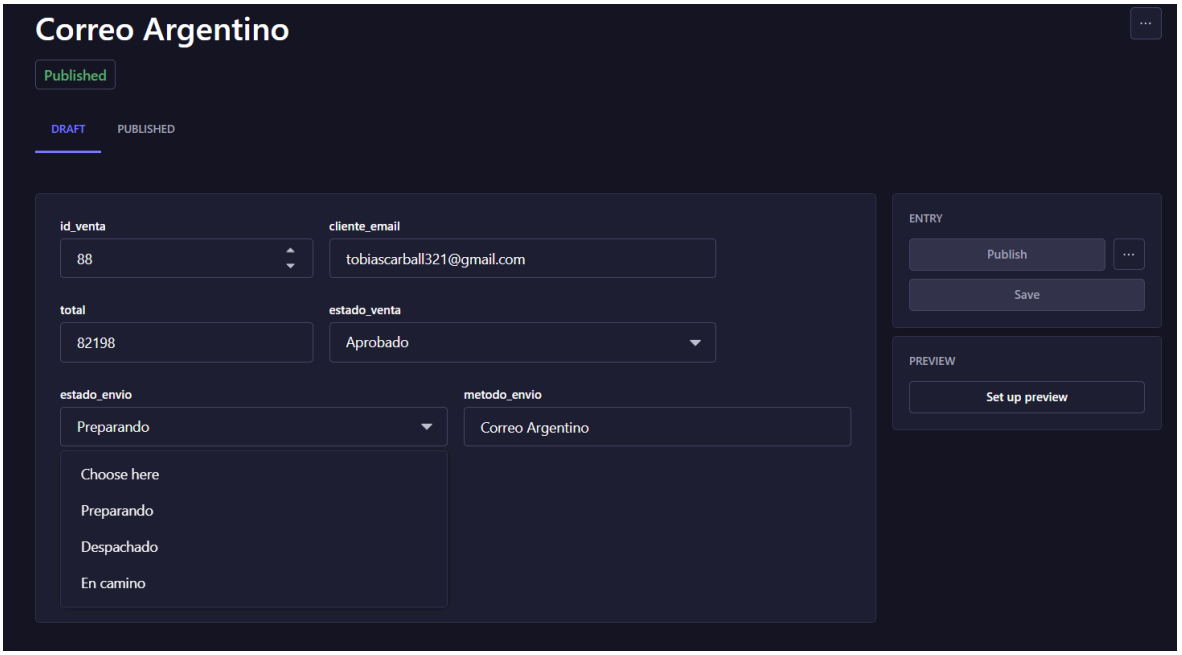



*Figura 46. Selector de estado dentro de un pedido.*

## MANUAL DEL DESARROLLADOR

Este manual describe los procedimientos necesarios para el despliegue, configuración y mantenimiento del ecosistema de software Mate Único. El sistema está compuesto por una arquitectura distribuida que incluye un CMS para gestión de contenidos, un servicio de autenticación y una interfaz de usuario.

**Prerrequisitos del entorno.**

Para garantizar la compatibilidad y el correcto funcionamiento de los servicios, el entorno de ejecución debe contar con:

- Node.js (v18 o superior): Motor de ejecución para los servicios de Frontend y Backend.
- PostgreSQL (v15 o superior): Motor de base de datos relacional para la persistencia de datos.
- Credenciales de Mercado Pago: Acceso al panel de desarrollador para la configuración de las claves de entorno (Access Tokens) en modo Sandbox.

**Configuración de la Base de Datos.**

La persistencia de Mate Único se gestiona mediante PostgreSQL. Siga estos pasos para la inicialización:

1. Creación de instancia: Genere una nueva base de datos denominada mate-unico.
2. Esquema y lógica: Localice el archivo SQL-principal.sql en la raíz del repositorio. Este script contiene la definición de:
    a. Tablas: Estructura de datos para productos, pedidos y usuarios.
    b. Dominios: Restricciones de tipos de datos personalizados.
    c. Triggers: Lógica automática para el control de integridad.

**Despliegue de los Servicios.**

El proyecto se divide en tres módulos independientes que deben ejecutarse simultáneamente:

**A. Servicio de autenticación (back-auth)**

Este módulo gestiona la seguridad, los tokens de acceso y la comunicación con la pasarela de pagos.

```
cd back-auth
npm install
# Configurar archivo .env con credenciales de DB y MP
npm run dev
```

**B. Gestor de contenidos (Strapi CMS)**

Encargado de la API de productos, cupones y administración general.

*cd back*
*npm install*
*npm run develop*

C. **Interfaz de usuario (front)**

Aplicación cliente desarrollada en React.

*cd front*
*npm install*
*npm run dev*

**Configuración de Variables de Entorno.**

Para la correcta comunicación entre módulos, asegúrese de configurar los archivos .env en las carpetas back-auth y back incluyendo:

- DATABASE_URL: Cadena de conexión a PostgreSQL.
- MP_ACCESS_TOKEN: Credencial privada de Mercado Pago.
- STRAPI_URL: Dirección de enlace con el CMS.

## INFORME DE CIERRE

**Cronograma Planificado y Cronograma Real.**

El proyecto presentó un desempeño superior a la planificación inicial. Mientras que la fecha estimada de finalización se situaba en junio de 2026, el equipo logró alcanzar el cierre técnico y funcional a fines de abril de 2026, logrando un adelanto de casi 5 semanas. A continuación, se presenta un cuadro comparativo con las fechas estimadas en la planificación y las fechas reales.

| Fase | Periodo Planificado | Finalización Real | Estado |
|---|---|---|---|
| Fase 1: Diseño técnico | Noviembre - Diciembre 2025 | Diciembre 2025 | Cumplido |
| Fase 2: Desarrollo Back-End | Febrero 2026 | Febrero 2026 | Cumplido |
| Fase 3: Fin Back - Comienzo Front | Marzo 2026 | Marzo 2026 | Cumplido |

| | | | |
|---|---|---|---|
| Fase 4: Desarrollo Front-End | Abril 2026 | Marzo 2026 | Adelantado |
| Fase 5: Integración y Pruebas | Mayo 2026 | Abril 2026 | Adelantado |
| Fase 6: Documentación Final | Junio 2026 | Abril 2026 | Adelantado |

El cumplimiento del cronograma con una antelación de aproximadamente 5 semanas respecto a la fecha planificada se fundamenta en la implementación de tres pilares de eficiencia operativa:

- Definición precisa del esquema de datos: Al invertir tiempo de calidad en el diseño técnico, se eliminaron ambigüedades técnicas. Esto permitió que, una vez iniciada la codificación en febrero, no surgieran cambios estructurales que suelen retrasar los proyectos.
- Paralelismo de tareas mediante arquitectura desacoplada: A diferencia de un desarrollo monolítico, la elección de una arquitectura basada en un CMS y un backend de autenticación independiente permitió trabajar en “carriles” paralelos. Este desacoplamiento evitó cuellos de botella, permitiendo que el desarrollo del frontend no tuviera que esperar a la finalización total del backend para comenzar la integración, además de una integración y desarrollo más rápido.
- Eficiencia del CMS: El uso de Strapi como motor de gestión de contenidos aceleró drásticamente los tiempos de desarrollo. Al proveer una interfaz de carga y una API REST funcional desde el primer momento, el equipo pudo dedicar este tiempo remanente a pulir la experiencia de usuario y la lógica de pago, tareas que originalmente se proyectaban como más extensas.

**Conclusión del Proyecto**

El objetivo principal se cumplió con éxito: desarrollar un ecosistema robusto que integra la flexibilidad de un CMS *Headless* (Strapi), la potencia de un *Frontend* reactivo (React) y la seguridad de un *Backend* (Node.js). La plataforma resultante va más allá de la funcionalidad de venta, ofreciendo una gestión de identidad de usuarios mediante Google y asegurando la seguridad de las transacciones a través de Mercado Pago.

**Desafíos Técnicos y Soluciones Implementadas**

El desarrollo de Mate Único presentó retos significativos que pusieron a prueba la arquitectura del sistema y la capacidad de resolución del equipo. Los principales desafíos fueron:

- Integración de la Pasarela de Pagos (Mercado Pago)
    - Desafío: Configurar el flujo de pago bajo estándares de seguridad profesional. El mayor obstáculo fue la gestión asíncrona de las notificaciones de pago. Durante el desarrollo local, el servidor de Mercado Pago no podía comunicarse con nuestra dirección localhost, lo que impedía validar si un pago se acreditaba correctamente para descontar el stock.
    - Solución: Se implementaron túneles mediante herramientas de exposición de puertos, permitiendo recibir señales externas en el entorno de desarrollo. Además, se desarrolló una lógica de fallback en el Backend que verifica el estado de la preferencia de pago antes de confirmar la orden, garantizando que el inventario solo se vea afectado tras una transacción exitosa.
- Autenticación con Google
    - Desafío: Lograr que el usuario pudiera registrarse e iniciar sesión de forma simplificada utilizando su cuenta de Google. Esto implicó

coordinar los tokens de Google con nuestra propia lógica de sesiones en el back-auth para mantener la persistencia del usuario en todo el ecosistema del sitio.

  - Solución: Se integró el SDK oficial de Google Auth en el Frontend, capturando el token de identidad y validándolo en nuestro servidor Node.js. Esto permitió crear automáticamente un perfil en nuestra base de datos PostgreSQL la primera vez que el usuario ingresa, vinculando su identidad social con su historial de compras en la tienda.

- Sincronización y Conexiones con Strapi (Headless CMS)
  - Desafío: Establecer una comunicación fluida y performante entre el Frontend (React) y el CMS. Al ser una arquitectura desacoplada, el reto fue manejar la estructura de datos anidada que devuelve Strapi y asegurar que la carga de imágenes y atributos (como el material del mate) no penalizará el tiempo de respuesta del sitio.
  - Solución: Se diseñó una capa de servicios en el Frontend utilizando fetch con manejo de estados globales. Se optimizaron las consultas mediante filtros específicos de la API de Strapi para traer únicamente los datos necesarios en cada vista, reduciendo el consumo de ancho de banda y mejorando la velocidad de carga de las galerías de productos.
- Integración de Datos y Persistencia de Pedidos
  - Desafío: Garantizar que toda la información crítica del pedido (productos, cantidades, personalizaciones y datos de envío) llegue de forma íntegra al administrador tras la confirmación del pago. Al interactuar con servicios externos (Mercado Pago), existía el riesgo de perder la trazabilidad de los detalles del carrito si la sesión del

usuario expiraba o si la conexión se interrumpía durante la redirección.

- Solución: Se implementó una lógica de persistencia previa. Antes de redirigir al usuario a la pasarela de pagos, el sistema genera una orden en estado "Pendiente" en PostgreSQL con un identificador único vinculado a la preferencia de Mercado Pago. Al procesar el pago, el backend recupera esta orden y vincula los datos del comprador con los ítems del carrito guardados, asegurando que el administrador reciba el detalle exacto del pedido (incluyendo el tipo de envío elegido) sin depender exclusivamente de la memoria del navegador del cliente.

**Trabajo Futuro: Hacia la Versión 2.0.**

El éxito de la versión actual sienta las bases para futuras mejoras orientadas a la expansión del negocio:

- Asistente Inteligente (AI Chatbot): Implementación de un chat basado en Inteligencia Artificial Generativa. Este asistente no solo resolverá dudas frecuentes sobre envíos o estados de cuenta, sino que actuará como un asesor de ventas personalizado. Mediante el análisis de las preferencias del usuario (materiales, tamaño del mate, tipo de yerba), la IA podrá recomendar el "combo" ideal para cada cliente, mejorando la conversión y la experiencia de usuario.
- Gestión Logística Avanzada: Integración con API de transportistas (ej. Correo Argentino/Andreani) para el cálculo de costos de envío y seguimiento en tiempo real.
- Dashboard de Analítica: Creación de un panel de métricas para el administrador que permita visualizar tendencias de venta y productos con mayor rotación.

- Reseñas Multimedia: Evolución del sistema de valoraciones para permitir que los clientes adjunten fotografías reales de los productos recibidos. Esta funcionalidad busca fortalecer la prueba social y la confianza de nuevos compradores al visualizar la calidad de los acabados y grabados en manos de otros usuarios.

**Cierre**

El desarrollo de Mate Único nos permitió resolver con éxito la totalidad de los desafíos técnicos planteados, pero fundamentalmente funcionó como un escenario de aprendizaje práctico sobre la integración de sistemas complejos en el mundo real. A través de este proyecto, logramos aprender la sincronización de servicios desacoplados, la gestión crítica de transacciones financieras y la importancia de una arquitectura limpia para cumplir con los objetivos antes de los plazos previstos. Esta experiencia consolidó nuestra capacidad para transformar requerimientos teóricos en una solución robusta y profesional, demostrando que la mayor lección fue aprender a coordinar diferentes tecnologías para que operen de forma transparente y eficiente para el usuario.